

생성모델

고려대학교 석준희

*ChatGPT: Optimizing
Language Models
for Dialogue*

We've trained a model called ChatGPT which interacts in a conversational way. The dialogue format makes it possible to challenge incorrect premises, and request more information. ChatGPT is a sibling model to GPT-3, which follows an instruction-based paradigm.



- 생성모델 개요
- VAE (Variational Autoencoder)
- GAN (Generative Adversarial Network)
- DDPM
- Score-based Generative Model
- D3PM

생성모델

생성모델 개요

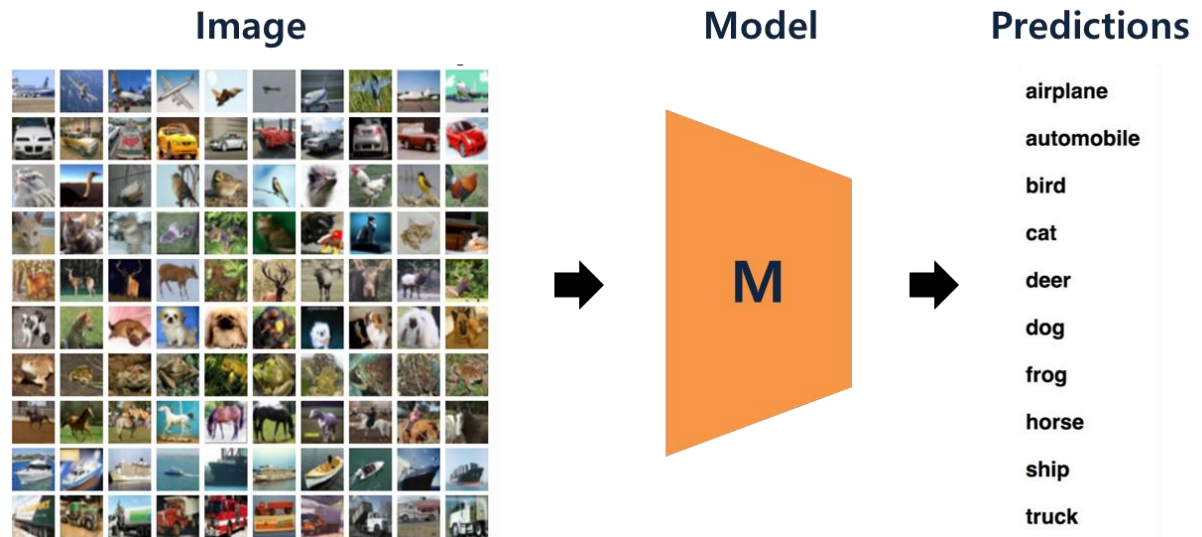


판별모델 vs. 생성모델

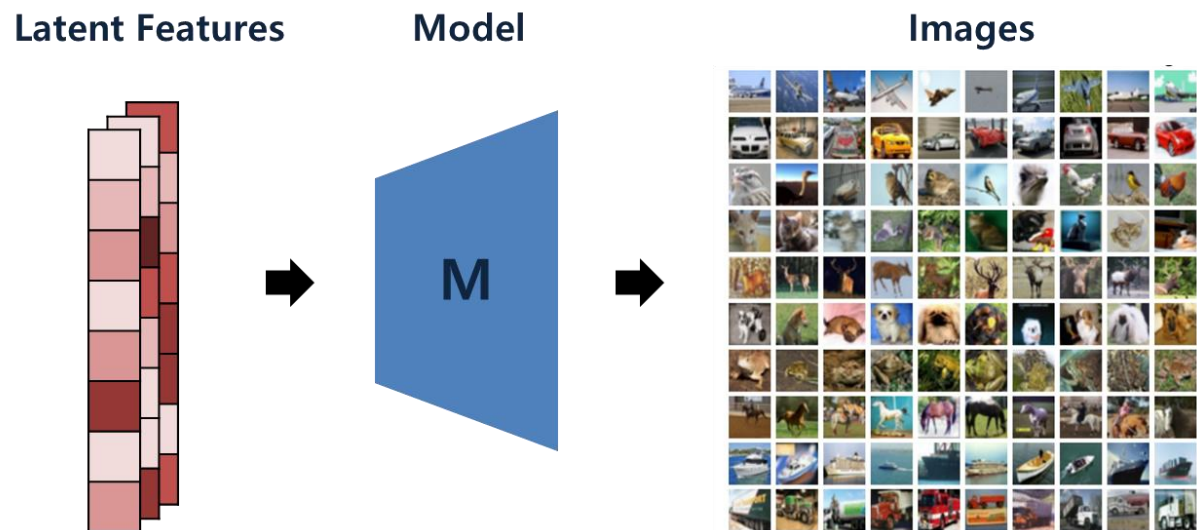
- **판별모델 (Determinative Model), 지도학습 모델**
 - 목적: 주어진 정보(x)로부터 관심 정보(y)를 결정
 - 데이터: (x, y) 의 쌍
 - 주요 문제: x 와 y 사이의 관계를 나타내는 함수 도출
 - 예: 분류, 회귀, 본 강의의 대부분의 모델들
- 비지도학습 모델
 - 목적: 주어진 정보(x)의 구조를 파악
 - 데이터: x (y 는 없음)
 - 주요 문제: x 의 확률 분포 혹은 숨겨진 구조를 파악
 - 예: 군집화, 차원축소 등
- **생성모델 (Generative Model)**
 - 목적: 주어진 정보(x)로부터 유사한 새로운 정보(x')를 생성
 - 데이터: x
 - 주요 문제: x 의 확률 분포를 파악하고 거기에서 임의의 샘플 생성
 - 예: 이미지 생성, 문장 생성 등

판별모델 vs. 생성모델

- CNN
판별모델



- 잠재변수
생성모델





생성모델의 활용

- 인간의 창조작업을 모방: 실제와 유사한 이미지, 아트웍, 작문 등



Berthelot et al, 2017



<https://pixabay.com/>



삶과 낭만을 주제로 200자 이내의 글을 작성해줘



삶은 우리가 살아가는 과정에서 경험하고 느끼는 때로는 고통스러울 수 있지만, 이 모든 경험들이 필요합니다. 그리고 그 과정에서 우리는 때로는 낭만적인 출을 보는 것일 수도 있고, 공원에서 책을 읽는 것 순간들은 우리 삶에 깊이 새겨지고, 우리를 더욱 에서 이러한 낭만적인 순간들을 만나며, 삶이 주는 시키며 새로운 것들을 배우며 살아가야 합니다.

- 데이터의 가공 및 변형: 고해상도화, 컬러화, 빈칸 채우기 등



<https://petapixel.com/>



Isola et al, 2017



Chen et al, 2017

- 데이터 확충, 가상 데이터 생성



생성모델의 활용

- ChatGPT (문장 생성) + StableDiffusion (이미지 생성)
- ChatGPT를 이용하여 이미지 생성에 필요한 문장(프롬프트) 생성
 - **프롬프트 (Prompt)**: 데이터 생성을 위해 인공지능에게 제공하는 문장



파워포인트 배경에 적합한 미래 기술적 이미지

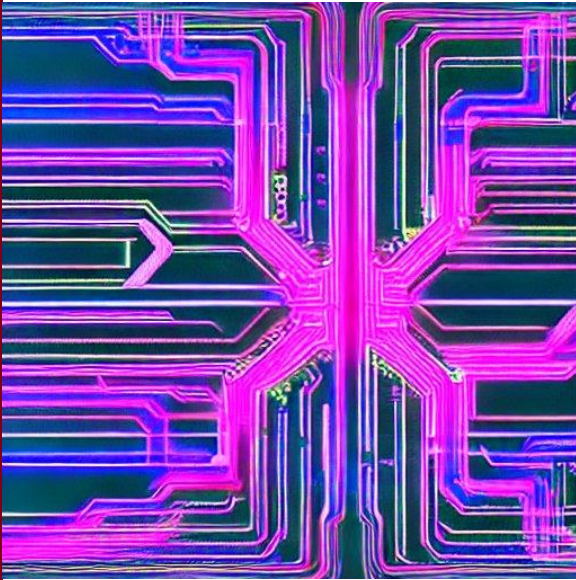


Here are a few suggestions for futuristic technological images that would be suitable for a PowerPoint background:

1. An abstract circuit board pattern with vibrant, neon-colored lines and circuits.
2. A high-tech city skyline with futuristic buildings and holographic displays.
3. A virtual reality headset or augmented reality glasses projecting holographic images.
4. A futuristic spaceship or spacecraft exploring distant planets or galaxies.
5. A futuristic laboratory with advanced equipment and holographic screens.
6. An AI robot or android interacting with humans in a futuristic setting.

생성모델의 활용

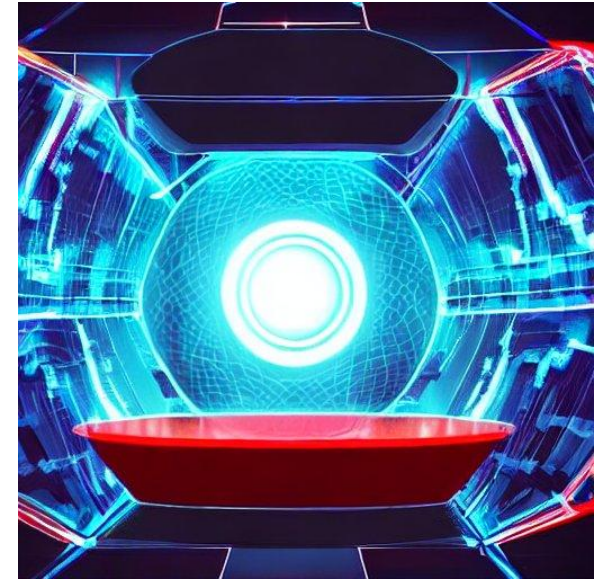
- StableDiffusion 으로 새로운 이미지 생성



futuristic technological images that would be suitable for a PowerPoint background. An abstract circuit board pattern with vibrant, neon-colored lines and circuits



futuristic technological images that would be suitable for a PowerPoint background. A high-tech city skyline with futuristic buildings and holographic displays



futuristic technological images that would be suitable for a PowerPoint background. A virtual reality headset or augmented reality glasses projecting holographic images



생성형 언어 모델

- 본래 생성모델은 이미지 생성을 위주로 발전
- 최근 ChatGPT와 함께 생성형 언어모델, 나아가 생성형 AI가 주목
- 생성형 언어모델
 - 언어모델: 단어(토큰)의 출현과 순서에 대한 확률을 나타내는 모델

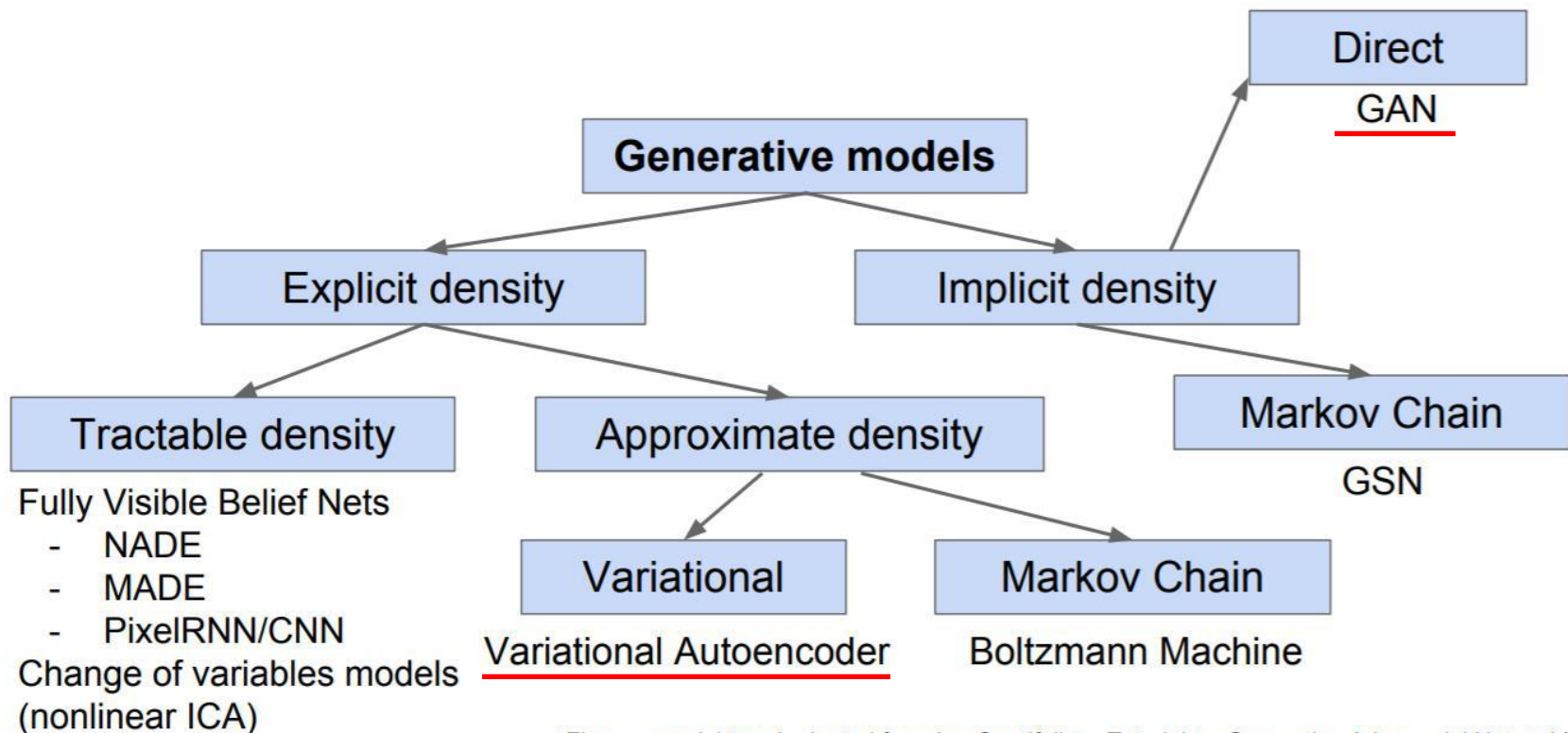
$$\begin{aligned}\text{Sentence: } w_0 w_1 \cdots w_t &\rightarrow \Pr[w_0, w_1, \dots, w_t] \\ &= \Pr[w_0] \Pr[w_1|w_0] \Pr[w_2|w_0, w_1] \cdots \Pr[w_t|w_0, w_1, \dots, w_{t-1}]\end{aligned}$$

- 생성형 모델로 활용 가능: 나는 학교에 <??> $\rightarrow \Pr[x| \text{'나는'}, \text{'학교에'}]$
 - ChatGPT의 경우 (사용자가 제시하는) 질문 뒤에 나올 문장을 예측하는 구조
- 본 강의에서는 이미지 생성 모델을 위주로 설명



[이미지] 생성모델의 종류

- **Explicit density:** 데이터의 확률분포를 학습하여 생성
 - 확률 분포 $\text{Pr}[X]$ 가 도출되고 여기에서 임의 추출을 통해 생성
- **Implicit density:** 데이터의 확률 분포를 명확히 학습하지 않고 생성
 - $\text{Pr}[X]$ 가 도출되지 않지만 모델이 내재하여 생성



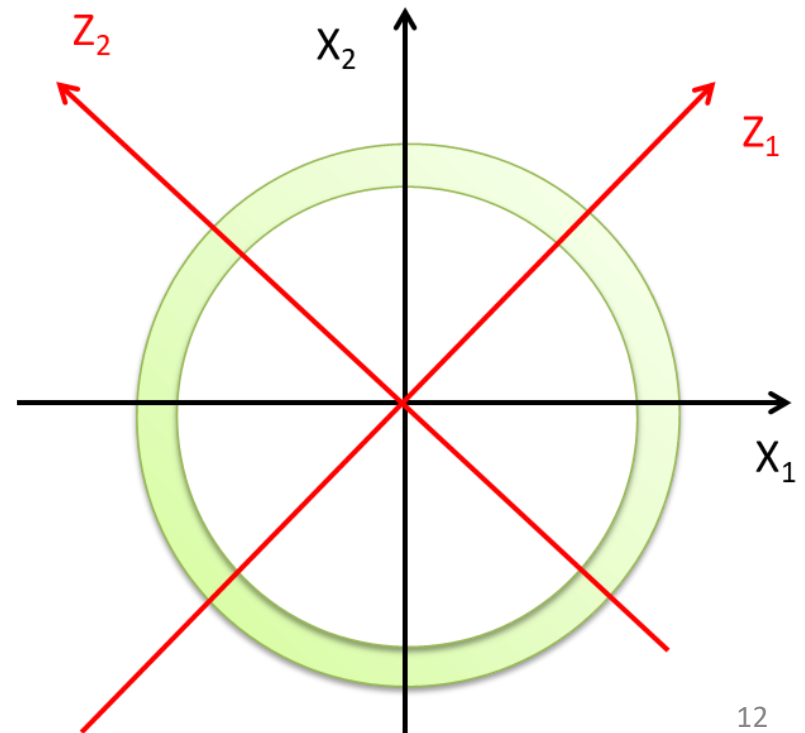
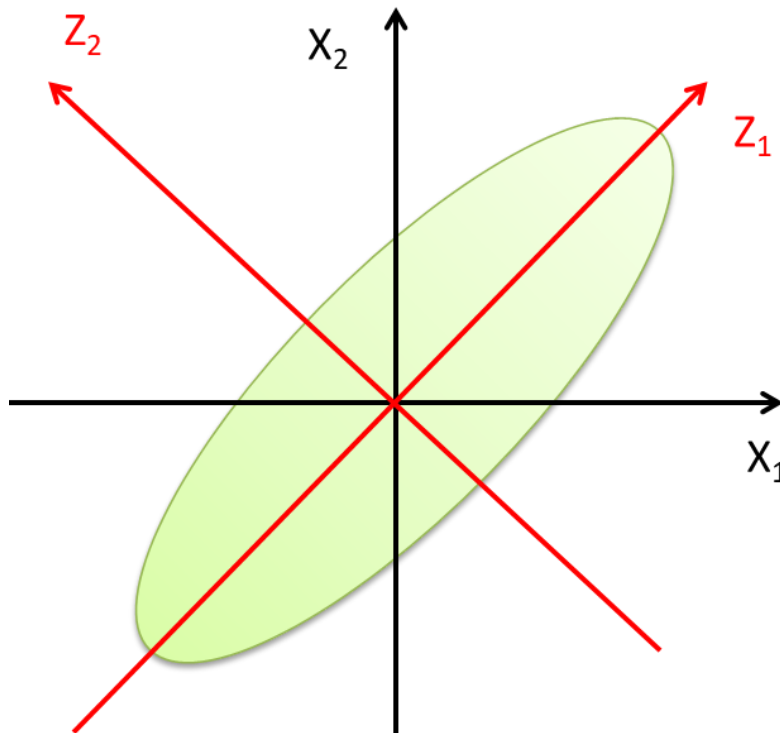
생성모델

VAE



차원 축소 (Dimension Reduction)

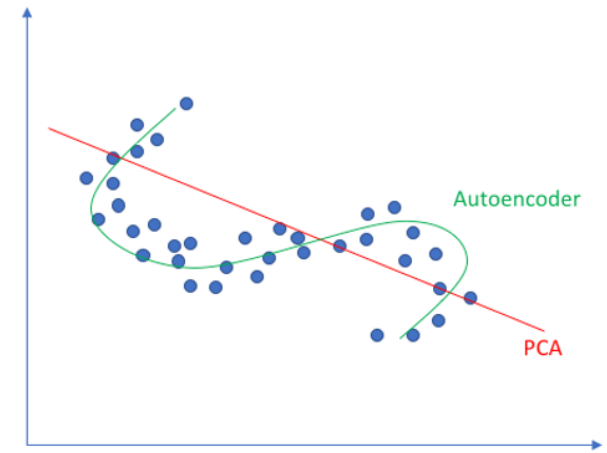
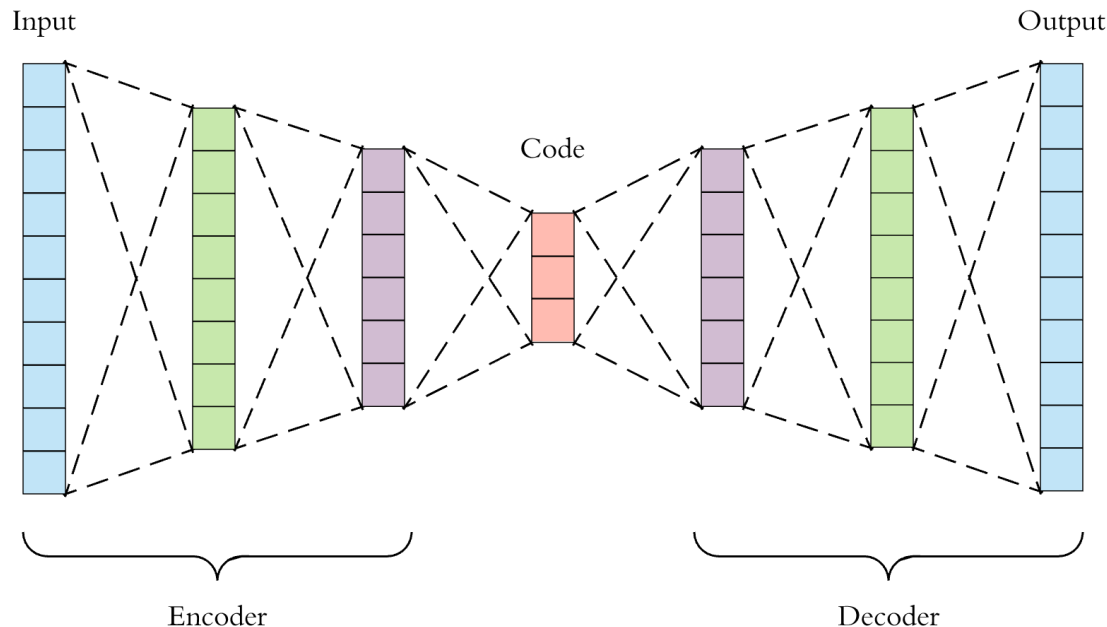
- 차원 축소: 주어진 p 차원의 입력 변수 공간 ($X_1, X_2, \dots, X_p$)를 더 낮은 k 차원의 공간($Z_1, Z_2, \dots, Z_k$)으로 변환 (강의 "모델의 선택과 확장" 참조)
- 주성분분석(PCA)
 - 선형변환이기 때문에 비선형 데이터에는 효과적이지 않음





AE: Autoencoder (오토인코더)

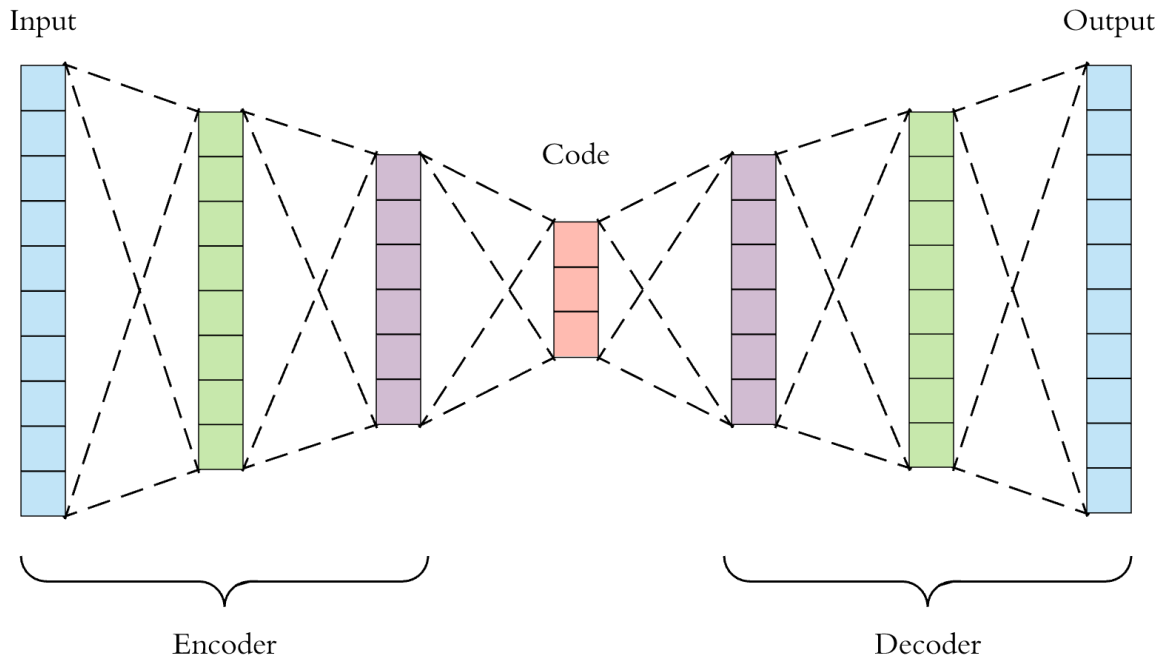
- **Autoencoder (AE):** 비선형적 차원축소를 위한 신경망 모델
- PCA의 학습
 - $X \rightarrow (\text{선형압축}) \rightarrow Z \rightarrow (\text{선형복원}) \rightarrow X'$: X 와 X' 의 차이를 최소화
- AE의 학습
 - $X \rightarrow (\text{비선형압축}) \rightarrow Z \rightarrow (\text{비선형복원}) \rightarrow X'$: X 와 X' 의 차이를 최소화
 - 비선형 압축/복원을 위해 신경망 구조를 사용





AE: Autoencoder (오토인코더)

- **Autoencoder (AE):** 비선형적 차원축소를 위한 신경망 모델
 - X : 관측변수, Z : 잠재변수
 - **Encoder:** 압축하는 신경망, $F(X) = Z$
 - **Decoder:** 복원하는 신경망, $G(Z) = X'$
 - 손실함수: $L = \|G(F(X)) - X\|^2$
- 신경망 구성
 - 일반적으로 인코더와 디코더는 대칭적으로 구성, MLP/CNN 등이 사용 가능



AE: Autoencoder (오토인코더)

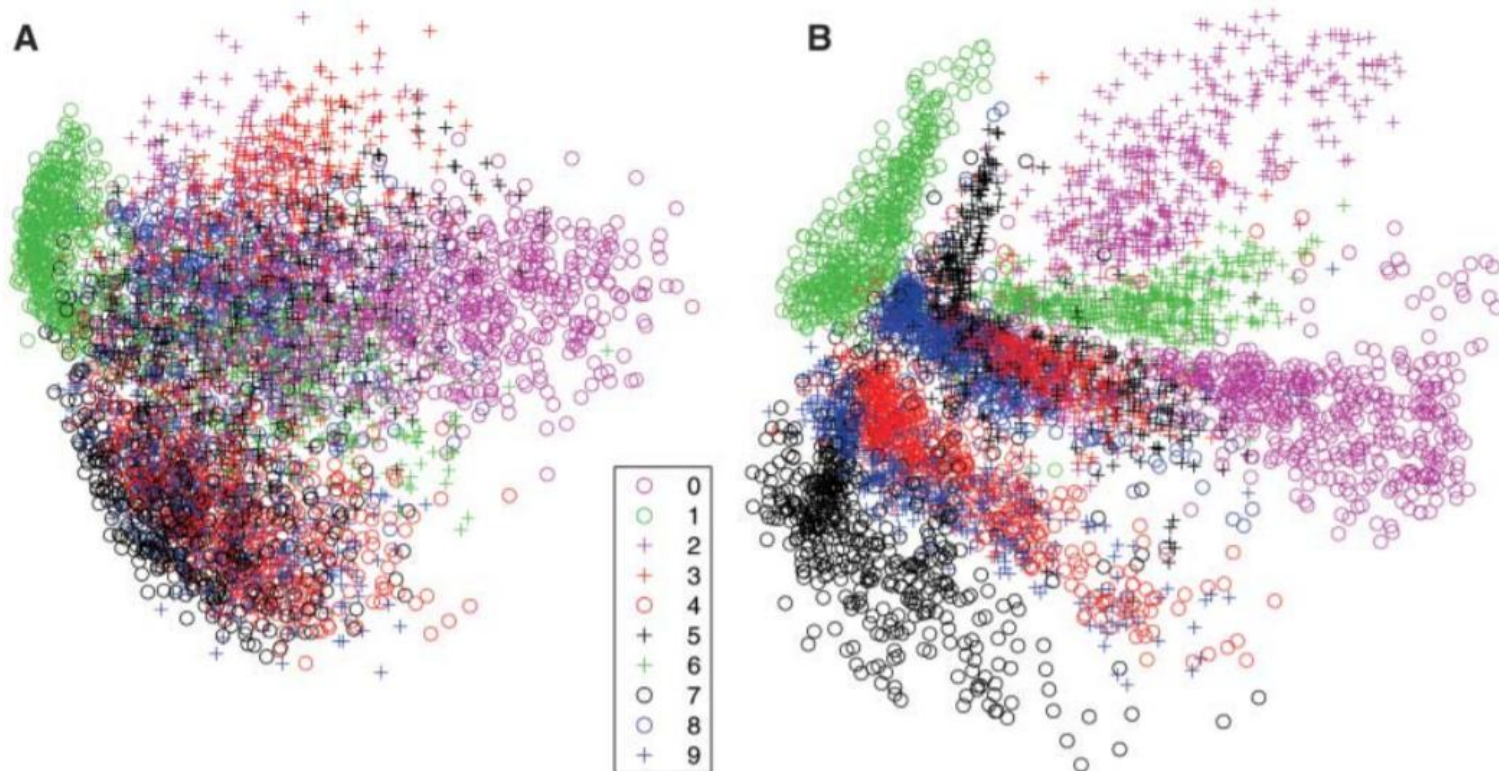
- 예시
 - MNIST data set
 - Hinton & Salakhutdinov, Science, 2006

Reducing the Dimensionality of Data with Neural Networks

G. E. Hinton* and R. R. Salakhutdinov

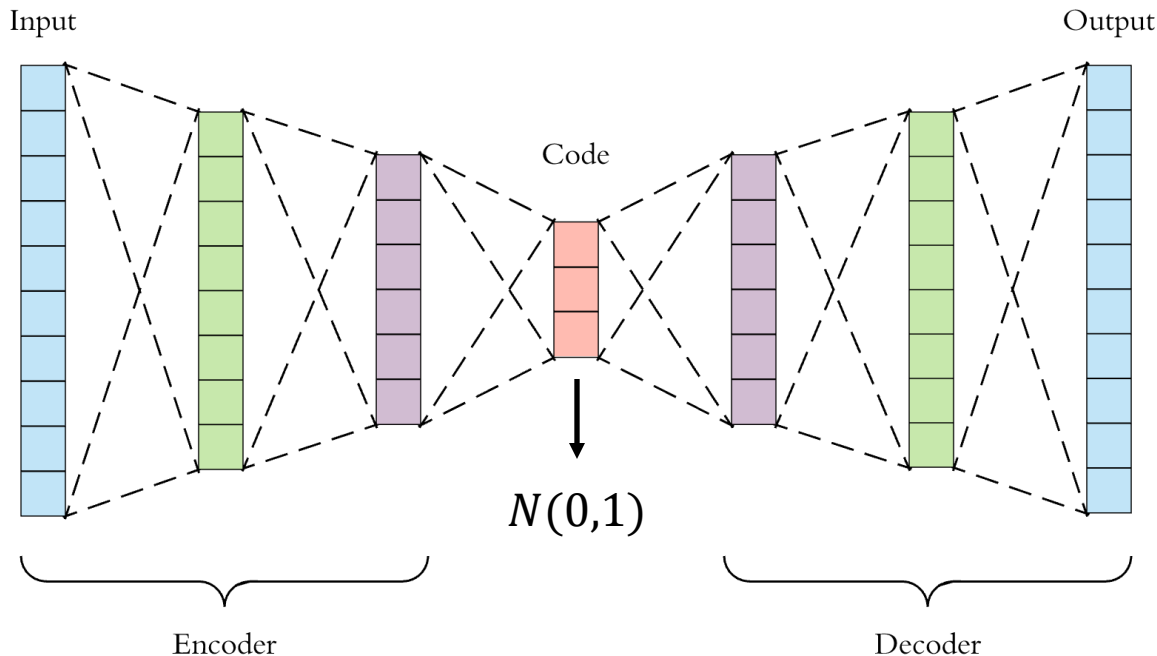
High-dimensional data can be converted to low-dimensional codes by training a multilayer neural network with a small central layer to reconstruct high-dimensional input vectors. Gradient descent can be used for fine-tuning the weights in such “autoencoder” networks, but this works well only if the initial weights are close to a good solution. We describe an effective way of initializing the weights that allows deep autoencoder networks to learn low-dimensional codes that work much better than principal components analysis as a tool to reduce the dimensionality of data.

Fig. 3. (A) The two-dimensional codes for 500 digits of each class produced by taking the first two principal components of all 60,000 training images. (B) The two-dimensional codes found by a 784-1000-500-250-2 autoencoder. For an alternative visualization, see (8).



VAE: Variational Autoencoder (변이형 오토인코더)

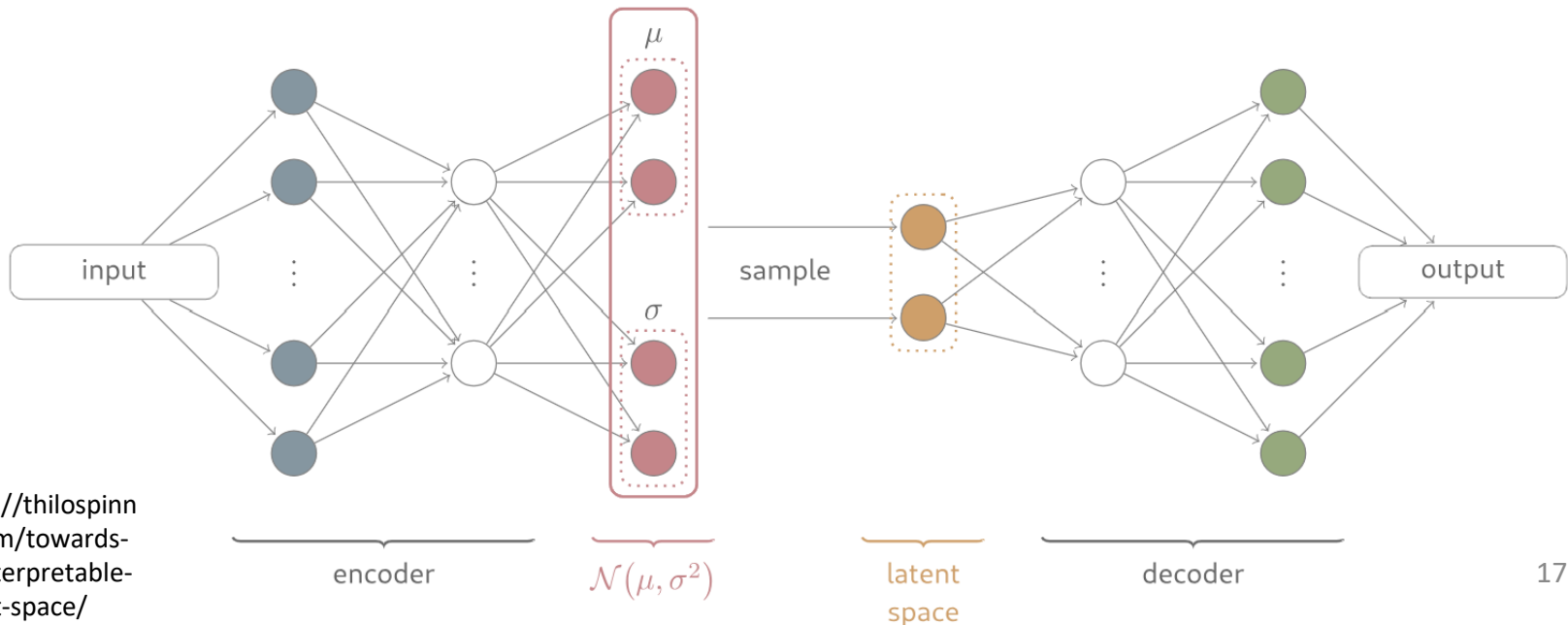
- AE를 사용한 새로운 데이터의 생성
 - 잠재공간에서 어떤 값이 주어지면 디코더를 통해 데이터의 생성이 가능
 - AE는 잠재변수의 확률 분포에 대해서는 관여하지 않음
 - 잠재공간에서 새로운 값을 추출하는 것이 어려움
- **VAE: Variational Autoencoder**
 - 잠재변수가 특정 확률 분포(e.g. 표준정규분포)를 갖도록 AE를 구성
 - 특정 확률 분포로부터 샘플을 생성하고, 디코더에 넣어 새로운 데이터 생성



VAE: Variational Autoencoder (변이형 오토인코더)

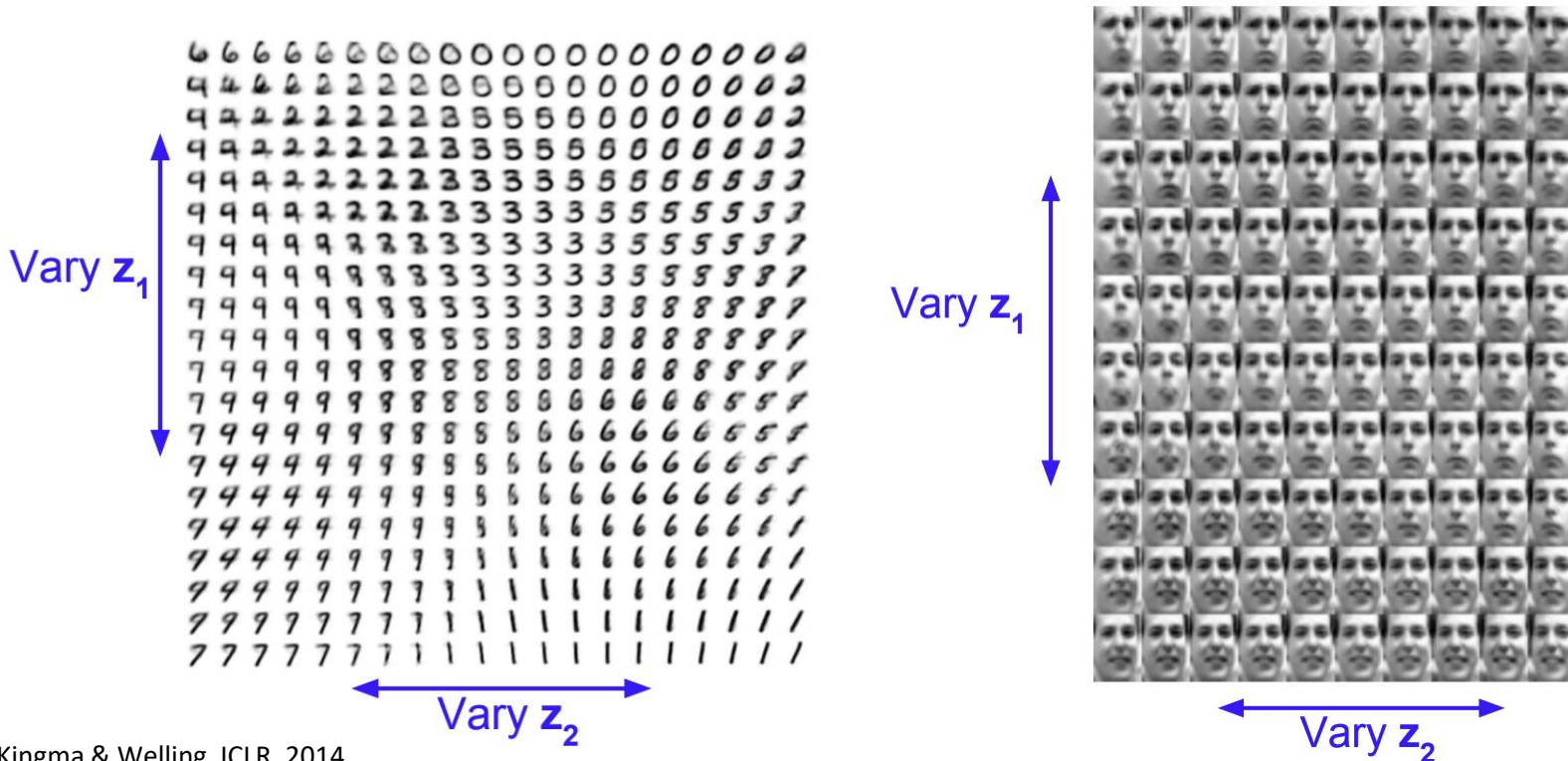
- 상세 구조
 - x 는 인코더를 통해 k -차원 평균(μ)과 표준편차(σ)의 두 잠재변수로 압축됨
 - $N(\mu, \sigma^2)$ 으로부터 잠재변수 z 를 임의로 추출
 - 디코더를 통해 z 로부터 x 를 다시 재구성
- 모델 훈련
 - $L = \|x - d(z)\|^2 + KL(N(\mu, \sigma^2), N(0,1))$
 - 같은 x 에 대해서 매번 다른 z 가 생성되어 $\hat{x}$ 을 생성

$N(\mu, \sigma^2)$: 실제 z 의 분포
 $N(0,1)$: 우리가 원하는 분포
 KL : Kullback-Leibler divergence,
두 분포 사이의 거리



VAE: Variational Autoencoder (변이형 오토인코더)

- 새로운 데이터 생성: 디코더에 $N(0,1)$ 에서 뽑힌 임의의 값을 넣어 생성

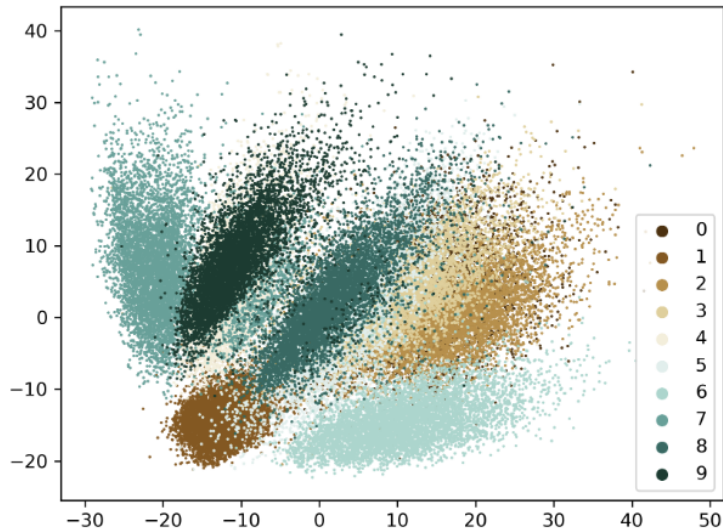


Kingma & Welling, ICLR, 2014

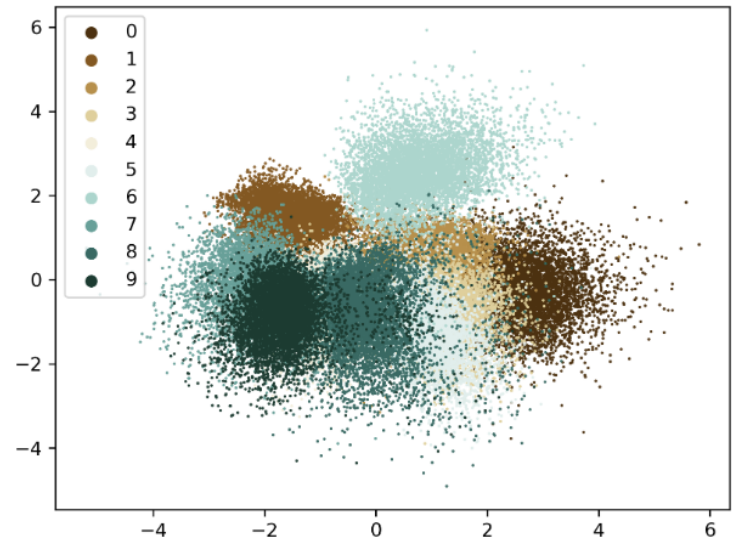
- 장점: 생성모델에 대한 수학적 접근과 확률분포 근사에 대한 접근법
- 단점: 다른 생성모델에 비해 성능이 떨어짐

VAE: Variational Autoencoder (변이형 오토인코더)

- AE vs. VAE
 - VAE가 좀더 밀집된 잠재공간을 갖고 있어 생성에 유리



(a) Latent Distribution by Label for AE



(b) Latent Distribution by Label for VAE

- VAE의 특징
 - 장점: 생성모델에 대한 수학적 접근과 확률 근사에 대한 접근법
 - 단점: 다른 생성모델에 비해 성능이 떨어짐

생성모델

GAN



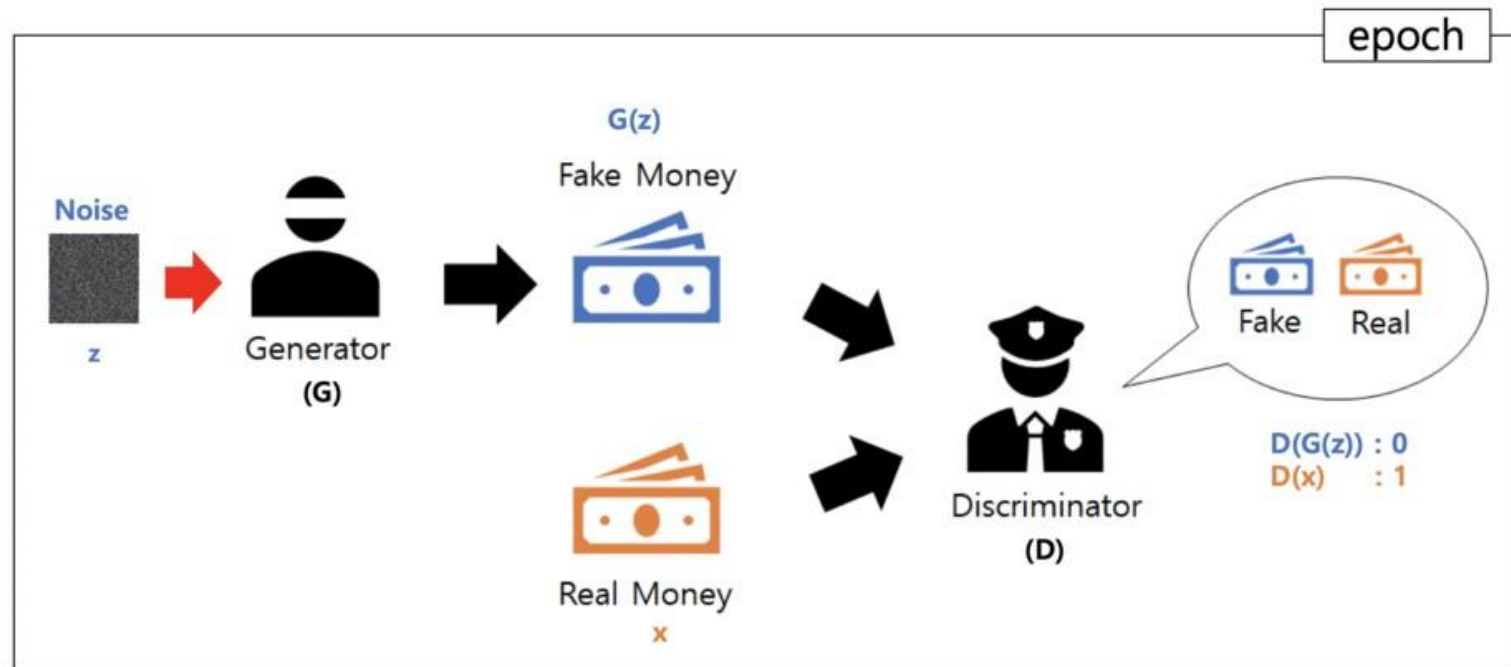
GAN: Generative Adversarial Network (적대적 생성모델)

- 고차원 데이터의 확률 분포를 찾는 것은 매우 어려운 문제
- 확률 분포를 찾지 않고도 데이터를 생성할 수 있을까?
- 게임 이론 (Game Theory)
 - 각 게임 참가자가 자신의 보상을 최대화 하기 위해 노력하는 상황에 대한 분석 (e.g. 죄수의 딜레마)
 - 경제학의 핵심 이론으로 출발하여 다양한 분야에 적용 중
- **GAN (Generative Adversarial Network)** [Goodfellow et al, NIPS, 2014]
 - 2인 게임 상황을 이용하여 새로운 데이터를 생성하는 모델
 - 생성자 (Generator): 실제와 유사한 새로운 데이터를 생성
 - 판별자 (Discriminator): 입력된 데이터가 실제 데이터인지 판별
 - 생성자와 판별자는 **적대적 관계**로 게임 속에서 서로 성장하는 구조



GAN: Generative Adversarial Network (적대적 생성모델)

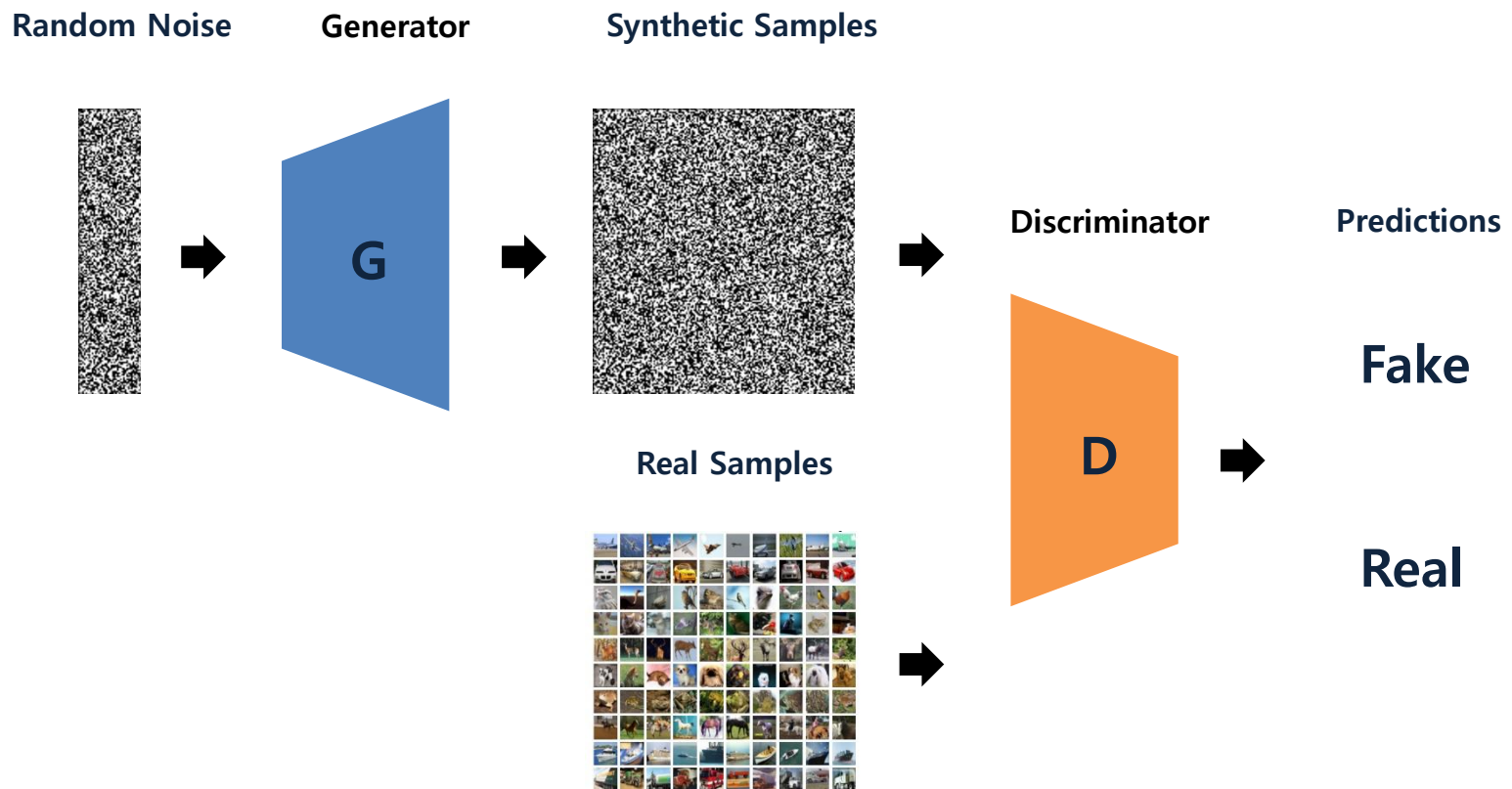
- **GAN (Generative Adversarial Network)**
 - 2인 게임 상황을 이용하여 새로운 데이터를 생성하는 모델
 - 생성자 (Generator, 범인): 실제와 유사한 새로운 데이터(위조지폐) 를 생성
 - 판별자 (Discriminator, 경찰): 입력된 데이터가 실제 데이터인지 판별 (위폐 판별)
 - 생성자와 판별자는 적대적 관계로 게임 속에서 서로 성장하는 구조





GAN 훈련 과정

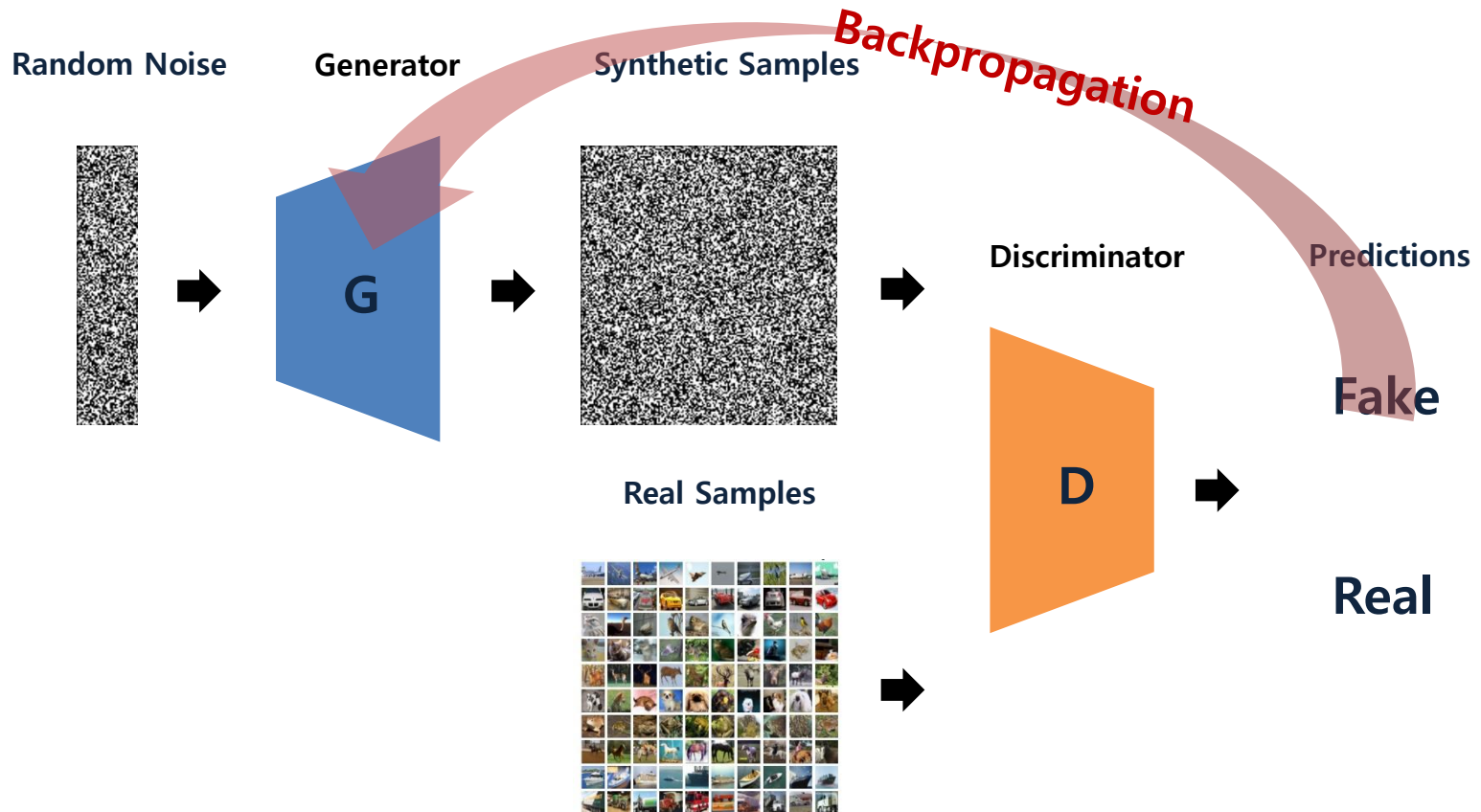
- 생성자와 판별자 사이의 적대적 관계를 통해 학습
- 초기에 생성자는 노이즈로부터 실제와 유사한 데이터를 생성
- 판별자를 이를 실제 데이터와 비교하여 실제인지 가짜인지 판단





GAN 훈련 과정

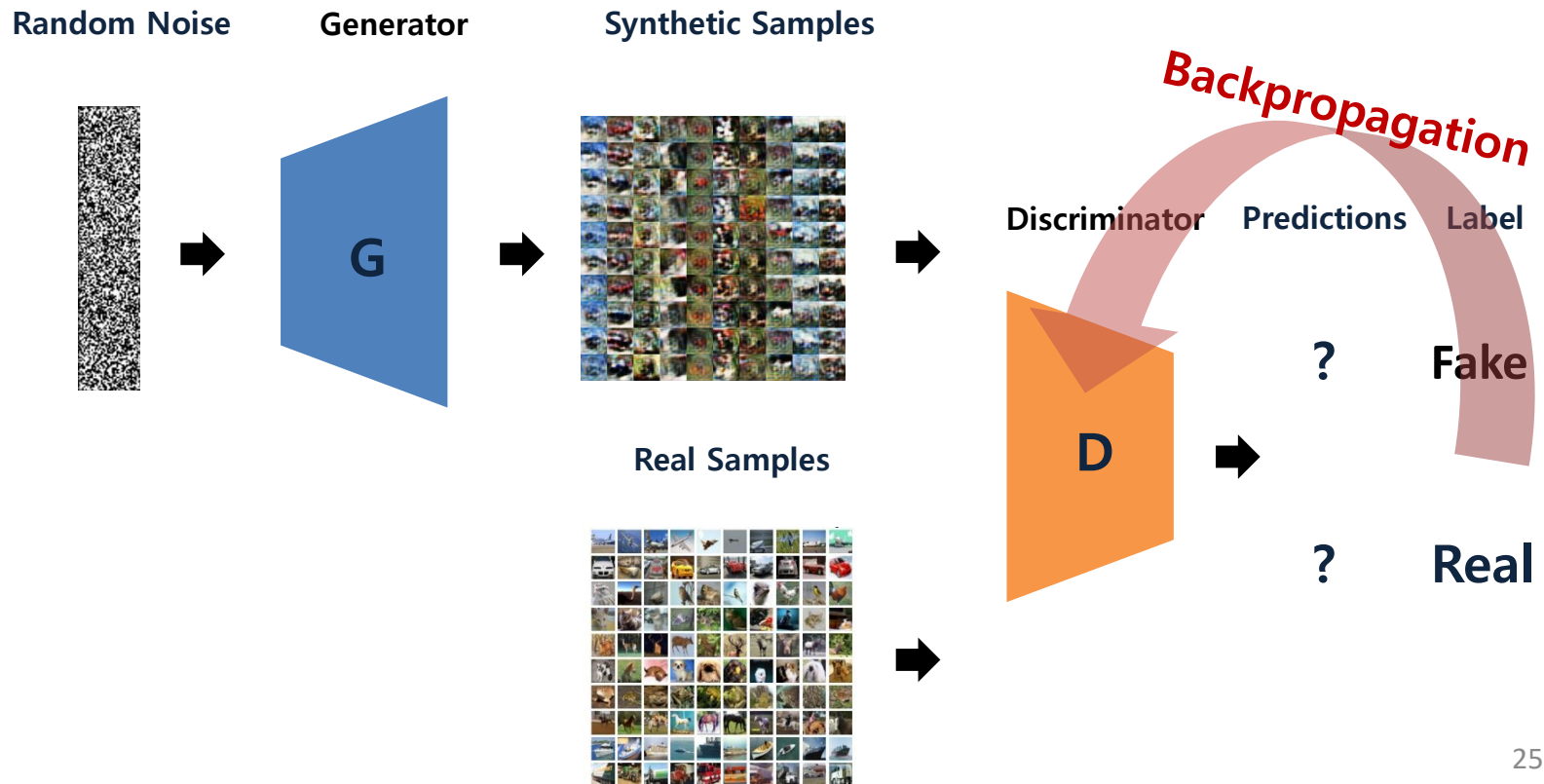
- 초기에 생성된 데이터는 구별이 쉽기 때문에 판별자가 쉽게 판단
- 이것을 손실함수로 하여 생성자를 학습
- 생성자는 판별자를 속일 수 있는 실제와 유사한 데이터를 생성하도록 학습





GAN 훈련 과정

- 생성자가 잘 훈련되어 실제와 유사한 데이터를 생성하면 판별자가 구별이 어려워져 에러가 증가
- 이것을 손실함수로 하여 판별자를 학습, 좀 더 잘 판별하도록 학습

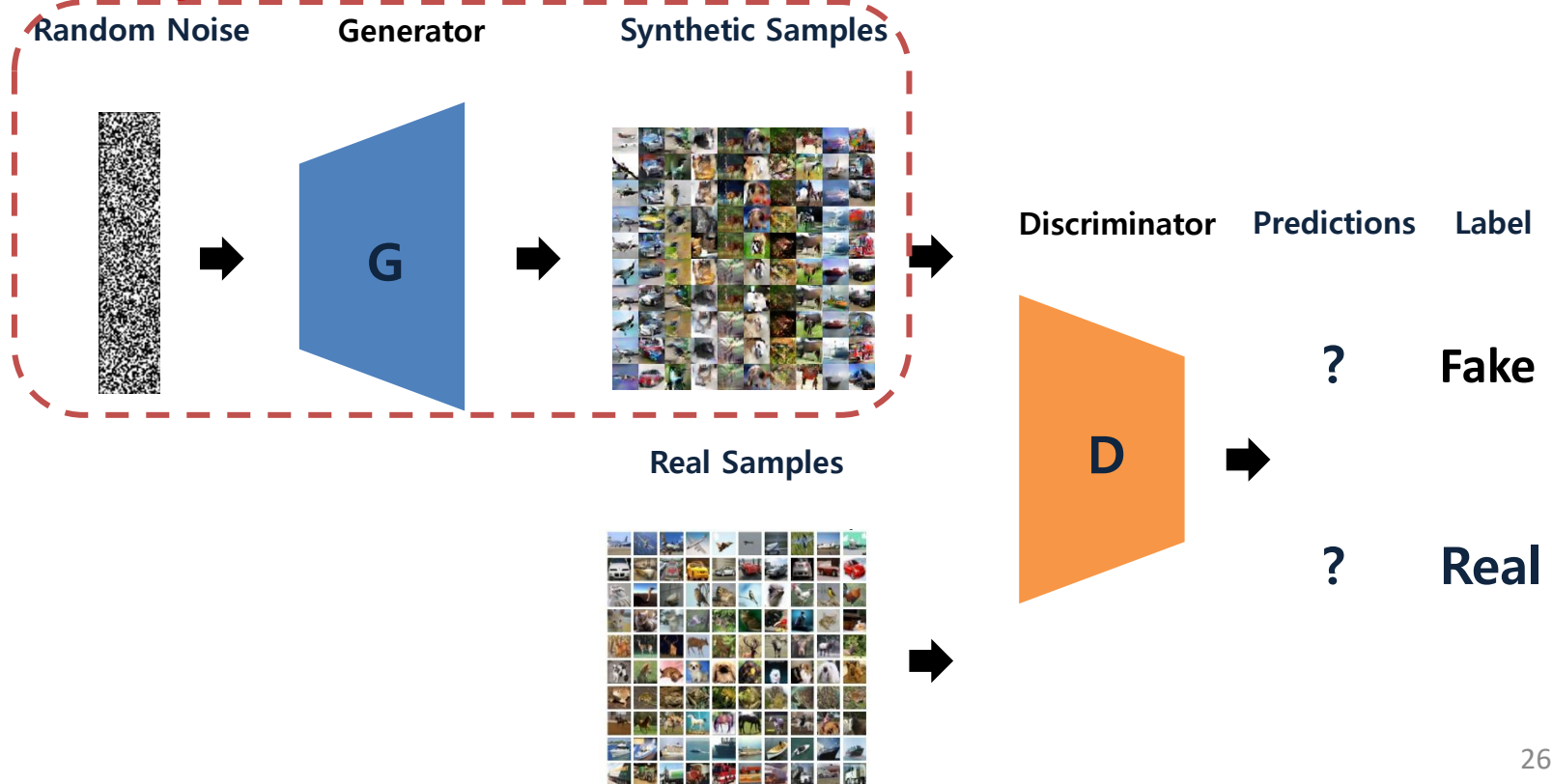




GAN 훈련 과정

- 학습이 잘 된 판별자는 더욱 생성 데이터를 구별 잘 하고, 이를 속이기 위해 생성자는 더욱 실제와 유사한 데이터를 생성
- 이 과정이 충분히 반복되면 생성자는 실제와 매우 유사한 데이터를 생성

Sample generation





GAN 훈련 과정

- x : 실제 데이터, z : 임의의 노이즈
- $D(x)$: 입력 x 에 대한 판별자의 판단 ($0 \sim 1$)
- $G(z)$: 입력 z 에 대하여 생성자에 의해 생성되는 데이터
- 판별자 손실함수: 판별자 학습에만 사용
 - $D(x)$ 는 1에 가까울수록, $D(G(z))$ 는 0에 가까울수록 손실이 감소

$$L_D = -\log D(x) - \log(1 - D(G(z)))$$

- 생성자 손실함수: 생성자 학습에만 사용
 - $D(G(z))$ 는 1에 가까울수록 손실이 감소

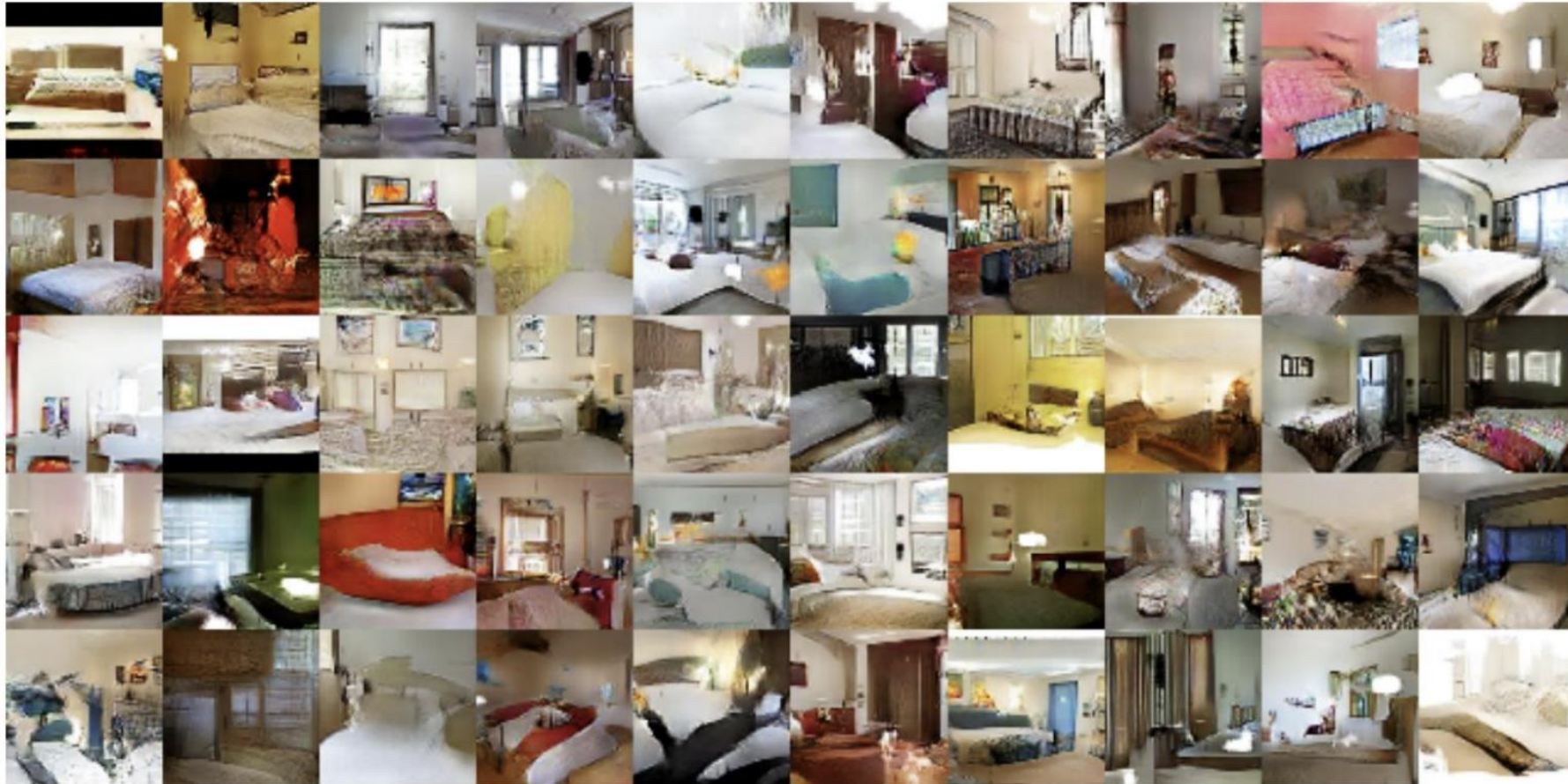
$$L_G = \log(1 - D(G(z)))$$

- 판별자와 생성자를 번갈아가며 학습을 진행
 - 실제로는 한쪽 손실이 너무 작아지면 다른 쪽과 상관없이 학습이 종료
 - 균형을 맞춰 학습하기가 어려움!



GAN 예시

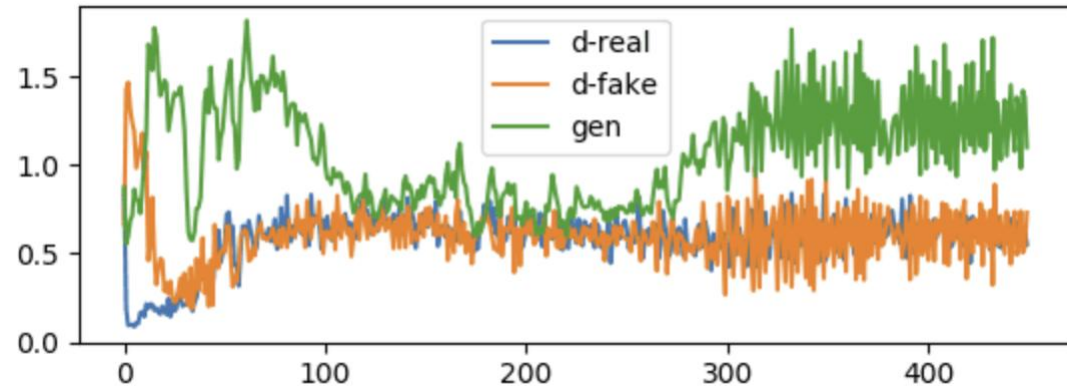
- 침실에 대한 이미지 생성 (Radford et al, ICLR, 2016)



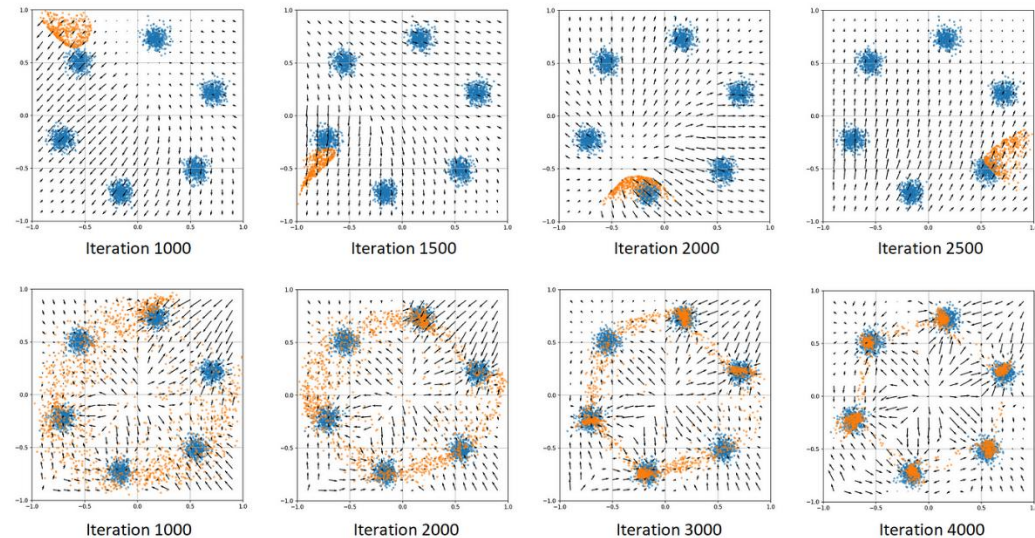


GAN의 문제점

- 훈련의 어려움
 - Oscillation
 - 손실이 계속적으로 감소하지 않고 진동
 - 경쟁구조이기 때문에 판별자와 생성자 모두 손실이 낮을 수 없음
- 다양성의 부족
 - Mode collapse
 - 생성자가 한 가지 유사한 데이터만 생성
 - 분포가 충분히 학습되지 않아 발생



<https://machinelearningmastery.com/practical-guide-to-gan-failure-modes/>



<https://pub.towardsai.net/gan-mode-collapse-explanation-fa5f9124ee73>



GAN의 발전

- 훈련을 안정화하기 위한 다양한 기법이 개발
- 빠르게 발전하여 실제와 같은 고해상도 이미지 생성도 가능



2014

GANs

2015

DCGAN

2016

CoGAN

2017

Progressive
Growing of GANs

2018

StyleGAN

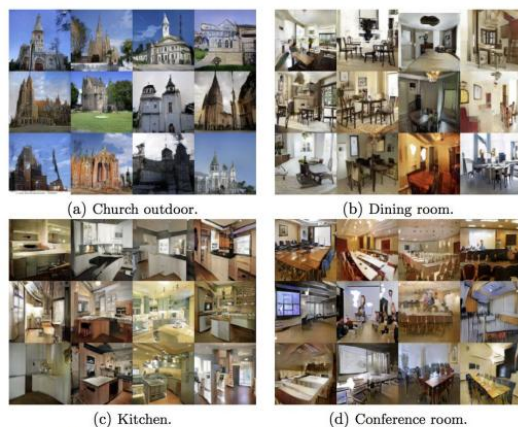
<https://wikidocs.net/146217>

GAN의 발전

- 다양한 응용

2017: Year of the GAN

Better training and generation

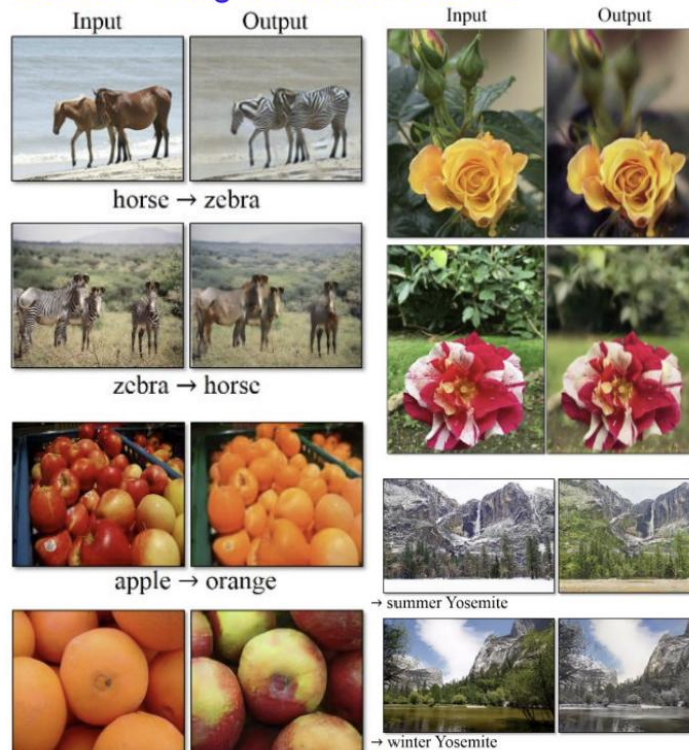


LSGAN. Mao et al. 2017.



BEGAN. Bertholet et al. 2017.

Source->Target domain transfer



CycleGAN. Zhu et al. 2017.

Text -> Image Synthesis

this small bird has a pink breast and crown, and black primaries and secondaries.



this magnificent fellow is almost all black with a red crest, and white cheek patch.



Reed et al. 2017.

Many GAN applications

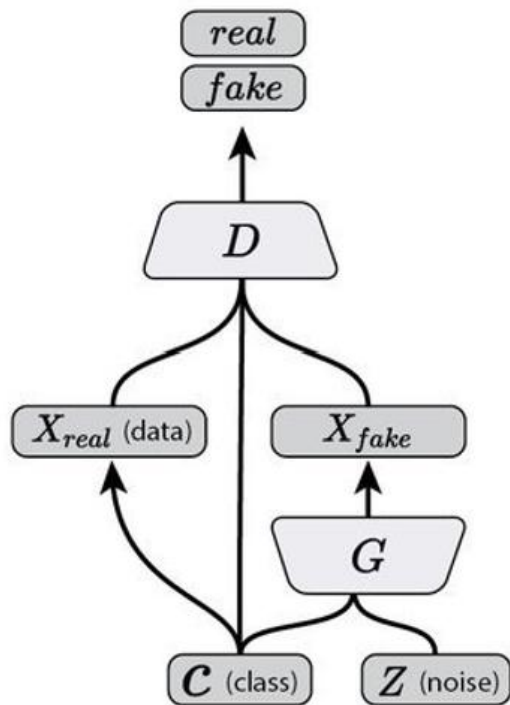


Pix2pix. Isola 2017. Many examples at <https://phillipi.github.io/pix2pix/>

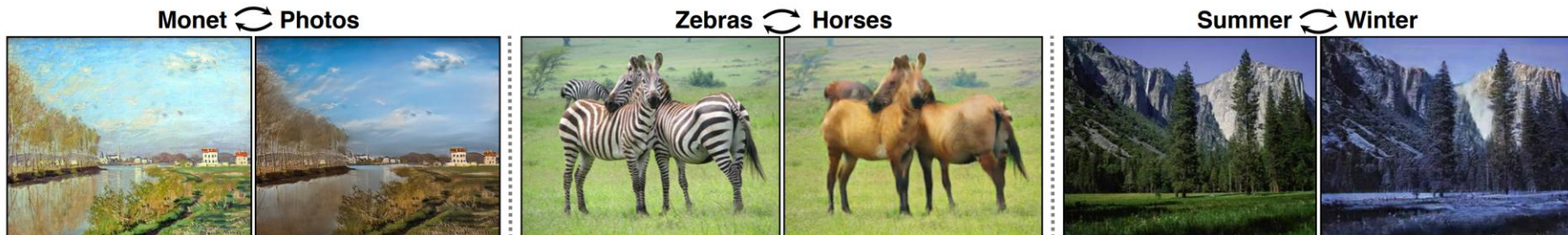


cGAN (Conditional GAN)

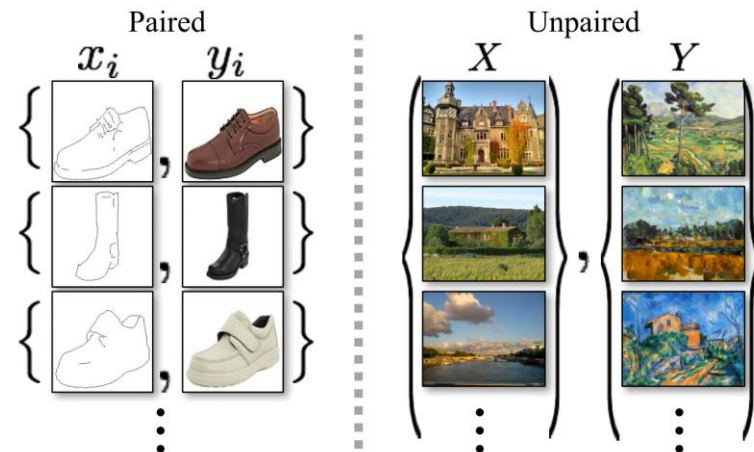
- 일반 GAN (Unconditional): 모든 데이터를 골고루 생성 (e.g. 0~9 모두 생성)
- cGAN**: 특정 조건(레이블)에 해당하는 데이터만 생성 (e.g. 8만 생성)
 - Mizra & Osindero, 2014



- **Style Transfer** (Image-to-image Translation)
 - 하나의 이미지를 다른 형식으로 변형하는 작업
 - 이미지 자체를 조건으로 주어 생성하는 cGAN의 형태로 구현 가능
 - 모델 훈련을 위해서는 일반적으로 paired 이미지가 필요 (e.g. 흑백-컬러)
 - 하지만 paired 이미지는 구하기가 어려움



- **CycleGAN**
 - Zhu et al, ICCV, 2017 (UC Berkeley)
 - Unpaired 이미지를 이용하여 style transfer를 구현



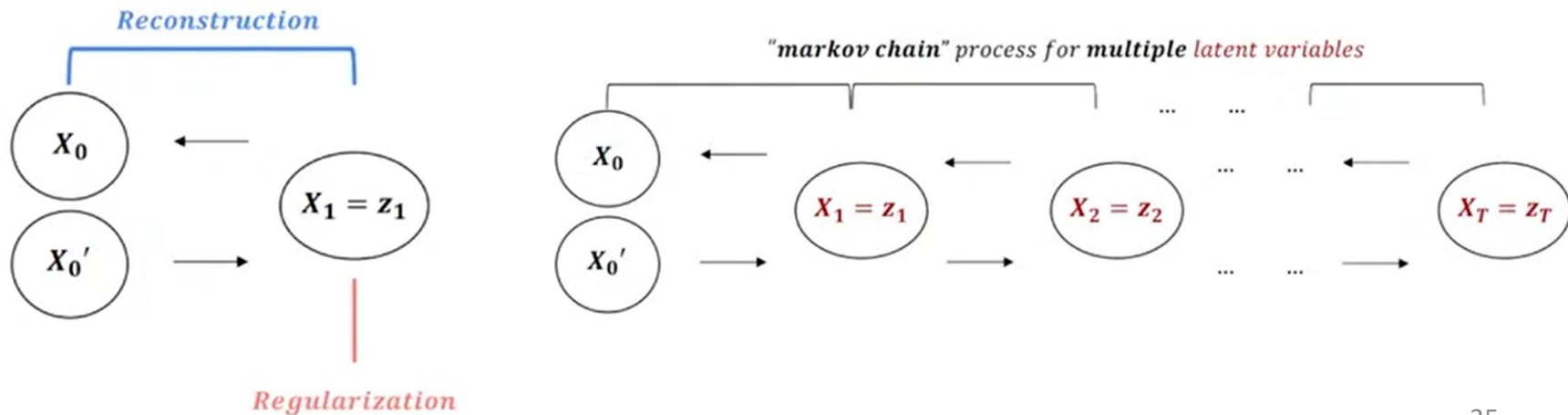
생성모델

DDPM



Diffusion 기본 아이디어

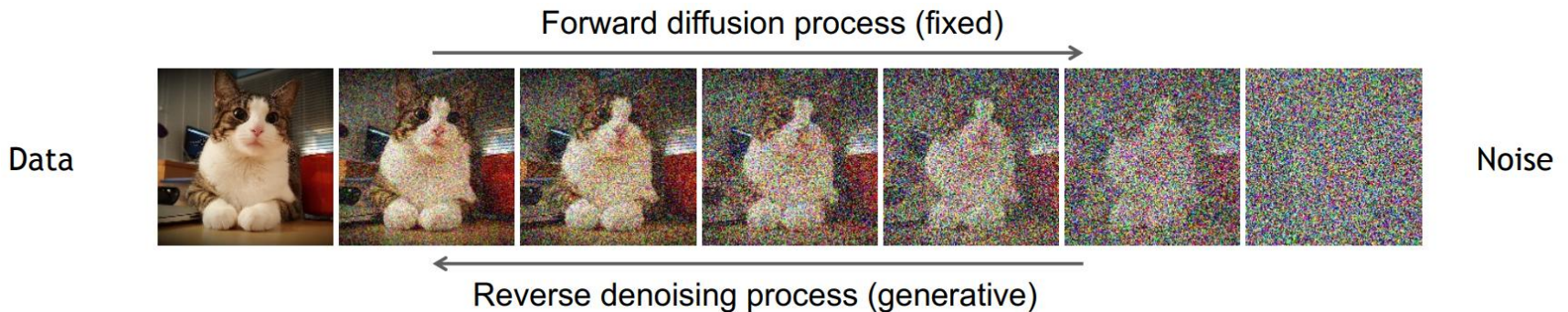
- VAE의 장단점
 - 장점: 다양한 이미지를 빠르게 생성하는 것이 가능
 - 단점: 고화질의 이미지 생성이 어려움
- GAN의 장단점
 - 장점: 고화질의 이미지를 빠르게 생성이 가능
 - 단점: 학습이 어렵고 다양한 이미지 생성이 어려움
- 고화질의 다양한 이미지를 얻을 수 있을까?
 - VAE를 여러 단계를 거쳐서 한다면??





Denoising Diffusion Probabilistic Model (DDPM)

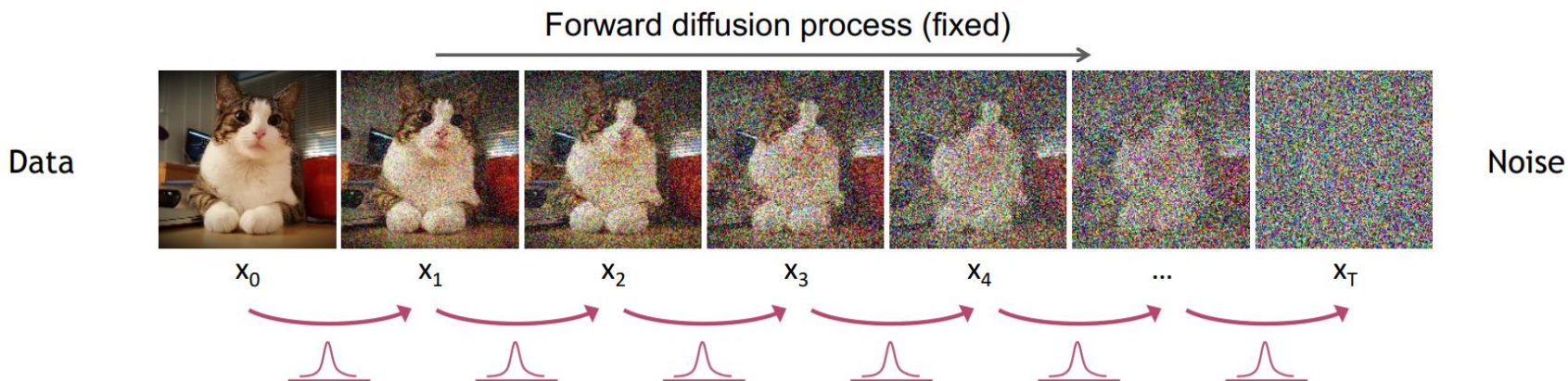
- DDPM은 2개의 프로세스로 구성
 - Forward diffusion process: 원래 데이터에 노이즈를 더해가는 과정
 - Reverse denoising process: 노이즈를 제거해가며 데이터를 생성하는 과정



[Sohl-Dickstein et al., Deep Unsupervised Learning using Nonequilibrium Thermodynamics, ICML 2015](#)
[Ho et al., Denoising Diffusion Probabilistic Models, NeurIPS 2020](#)
[Song et al., Score-Based Generative Modeling through Stochastic Differential Equations, ICLR 2021](#)

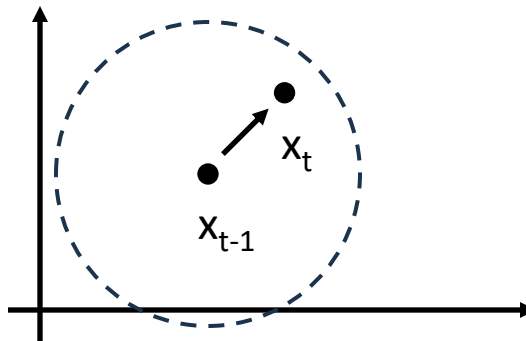


Forward Diffusion Process



$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \underbrace{\sqrt{1 - \beta_t} \mathbf{x}_{t-1}}_{\text{평균}}, \underbrace{\beta_t \mathbf{I}}_{\text{분산}}) \quad \rightarrow \quad q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1}) \quad (\text{joint})$$

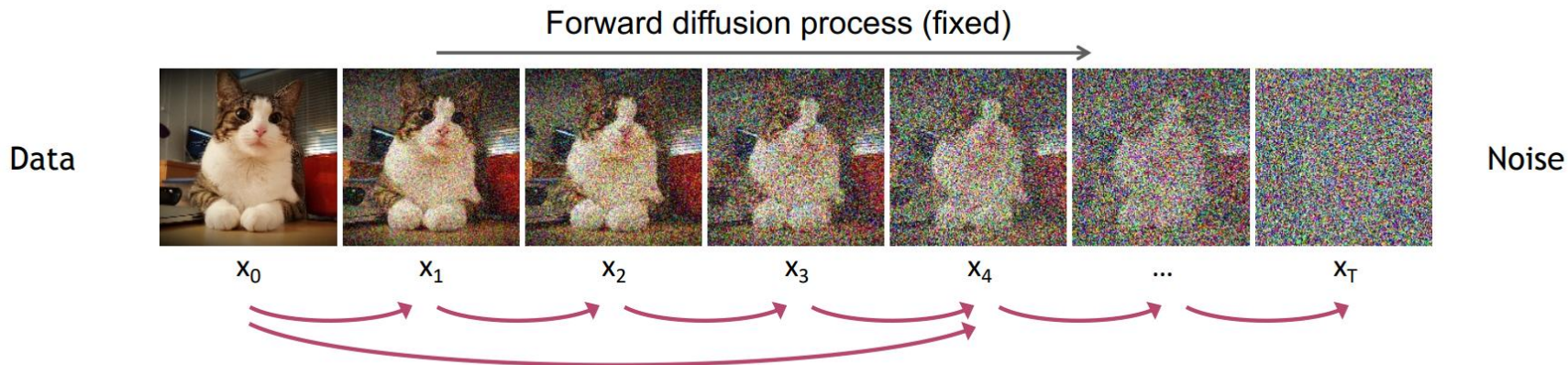
$\nwarrow$
 $\mathbf{x}_{t-1}$ 에서 $\mathbf{x}_t$ 로 넘어가는데
 필요한 노이즈의 분포



$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon$$



Forward Diffusion Process



Define $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$ $\rightarrow$ $q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$ (Diffusion Kernel)

For sampling: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{(1 - \bar{\alpha}_t)} \epsilon$ where $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

↙ $\mathbf{x}_t$ 를 얻기 위해서 t번 노이즈를 더할
필요없이 한 번에 얻을 수 있음

β_t values schedule (i.e., the noise schedule) is designed such that $\bar{\alpha}_T \rightarrow 0$ and $q(\mathbf{x}_T | \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

보통은 점점 증가하는 값을 사용: 0.0001 -> 0.02



Generative Learning by Denoising

Generation:

Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$

Iteratively sample $\mathbf{x}_{t-1} \sim \underbrace{q(\mathbf{x}_{t-1}|\mathbf{x}_t)}_{\text{True Denoising Dist.}}$

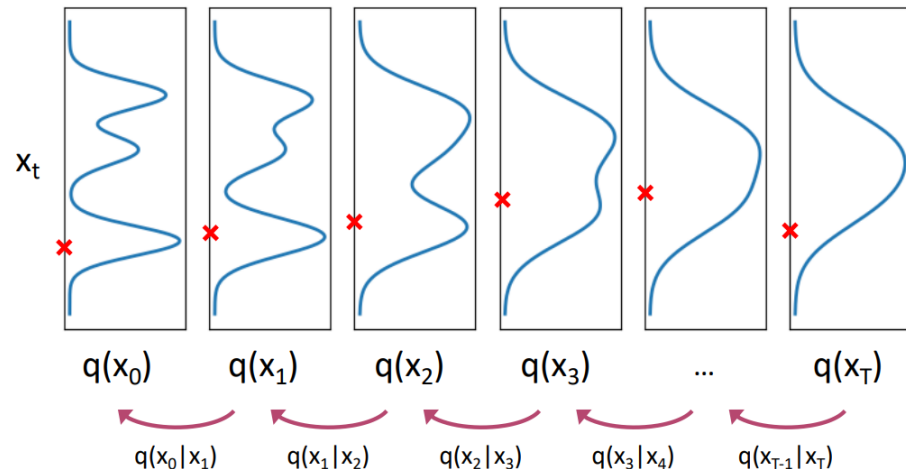
$$q(\mathbf{x}_T) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$\mathbf{x}_t = \sqrt{1 - \beta_t} \mathbf{x}_{t-1} + \sqrt{\beta_t} \epsilon$$



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

Diffused Data Distributions

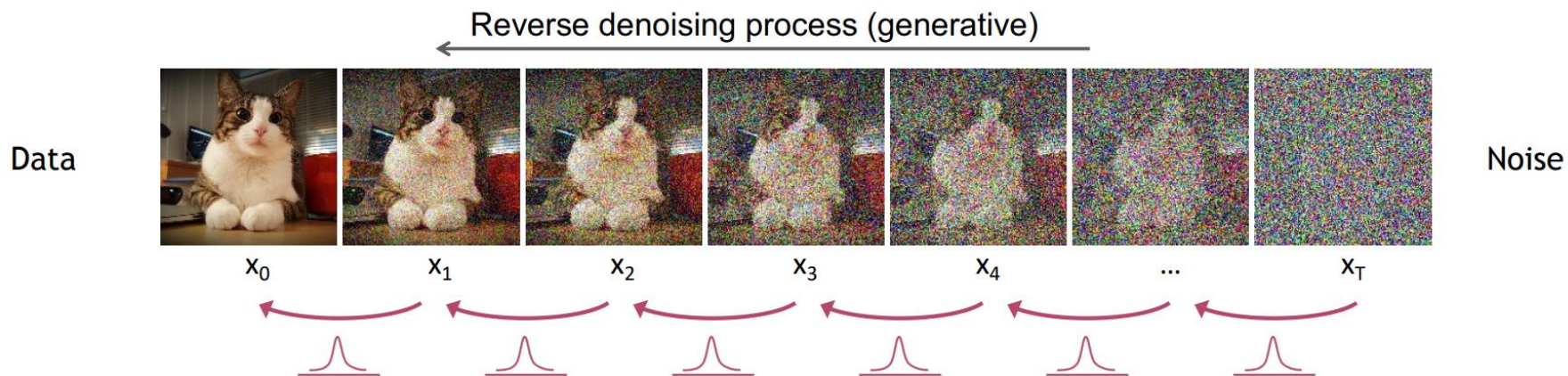


$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = N\left(\frac{\mathbf{x}_t}{\sqrt{1-\beta_t}}, \left(\frac{\sqrt{\beta_t}}{\sqrt{1-\beta_t}}\right)^2 \mathbf{I}\right) ???$$

실제 분포는 Normal이 아니지만
Beta가 작다면
Normal에 가까울 것으로 예상



Reverse Denoising Process

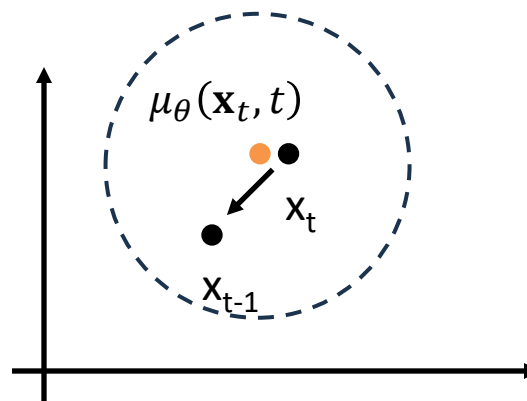


$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \underbrace{\mu_{\theta}(\mathbf{x}_t, t)}_{\text{Trainable network}}, \sigma_t^2 \mathbf{I}) \quad \rightarrow \quad p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Trainable network
(U-net, Denoising Autoencoder)

$q(x)$: x 의 실제 분포
 $p(x)$: 모델이 예측한 분포





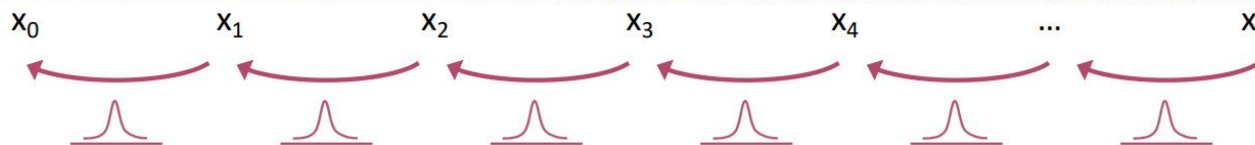
Reverse Denoising Process

Reverse denoising process (generative)

Data



Noise



$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$

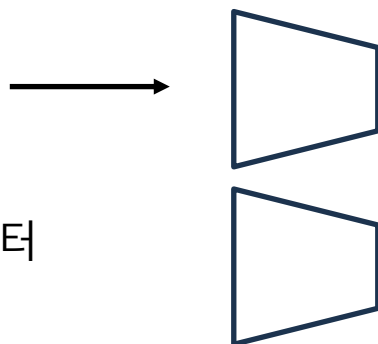
$$p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \underbrace{\mu_{\theta}(\mathbf{x}_t, t)}_{\text{Trainable network}}, \sigma_t^2 \mathbf{I}) \quad \rightarrow \quad p_{\theta}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1}|\mathbf{x}_t)$$

Trainable network
(U-net, Denoising Autoencoder)

$$\mathbf{x}_{t-1} = \frac{\mathbf{x}_t - \sqrt{\beta_t} \boldsymbol{\epsilon}}{\sqrt{1 - \beta_t}}$$



노이즈가 섞인 데이터



$\mu_{\theta}(\mathbf{x}_t, t)$ 이전 단계 데이터의 평균

$\epsilon_{\theta}(\mathbf{x}_t, t)$ 빼주어야 할 노이즈의 평균



Reverse Denoising Process

원본 데이터에서 적당히 노이즈가 섞인 데이터 생성

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

이전 단계의 평균과 빼 주어야 할 노이즈 사이의 관계

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) \quad \bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$$

손실 함수의 표현 (Upper bound를 최소화)

$$\mathbb{E}_{q(\mathbf{x}_0)} [-\log p_{\theta}(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[-\log \frac{p_{\theta}(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L \quad \text{최대값}$$

훈련 데이터를 관측할 확률

$$L = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), t \sim \mathcal{U}\{1, T\}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\lambda_t || \epsilon - \epsilon_{\theta}(\underbrace{\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon}_{\mathbf{x}_t}, t) ||^2 \right]$$

Summary

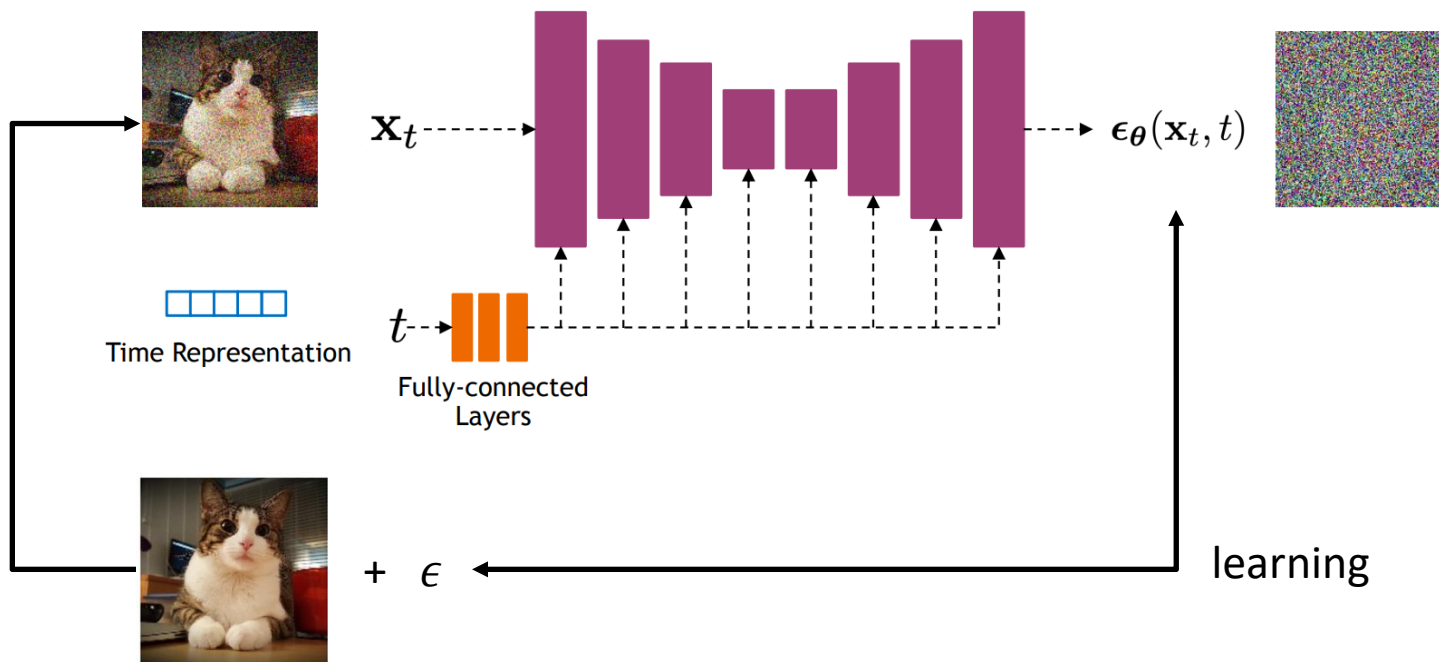
Algorithm 1 Training

- 1: **repeat**
- 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- 3: $t \sim \text{Uniform}(\{1, \dots, T\})$
- 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 5: Take gradient descent step on

$$\nabla_{\theta} \left\| \epsilon - \epsilon_{\theta}(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right\|^2$$
- 6: **until** converged

Algorithm 2 Sampling

- 1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 2: **for** $t = T, \dots, 1$ **do**
- 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
- 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
- 5: **end for**
- 6: **return** $\mathbf{x}_0$



생성모델

Score-based Generative Model



Score Matching

- Score function: 어떤 확률 분포 $p(x)$ 의 로그값의 gradient

$$\nabla_{\mathbf{x}} \log p(\mathbf{x})$$

- 보통 확률 분포를 모델을 통해 추정할 때, 스케일링 팩터를 구하기 어려운 경우가 많음

$$p_{\theta}(\mathbf{x}) = \frac{e^{-f_{\theta}(\mathbf{x})}}{Z_{\theta}}, \text{ where } Z_{\theta} = \int e^{-f_{\theta}(\mathbf{x})}$$

우리가 잘 아는 함수 형태

계산이 어려움

- 이럴 때 score function을 이용하면 쉽게 계산이 가능 (스케일팩터는 x 에 독립)

$$\nabla_{\mathbf{x}} \log p(\mathbf{x}) = -\nabla_{\mathbf{x}} f_{\theta}(\mathbf{x})$$



Score Matching

- 원래 풀어야하는 문제

$$E_{p_{data}}[\|p_{data}(\mathbf{x}) - p_{\theta}(\mathbf{x})\|_2^2] \longrightarrow \text{어려움}$$

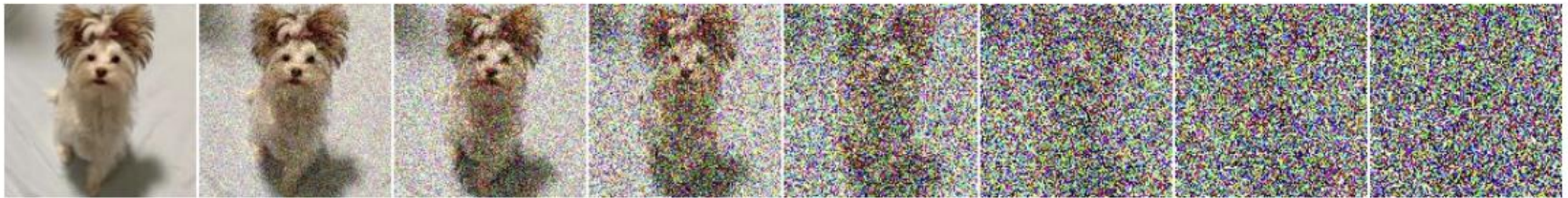
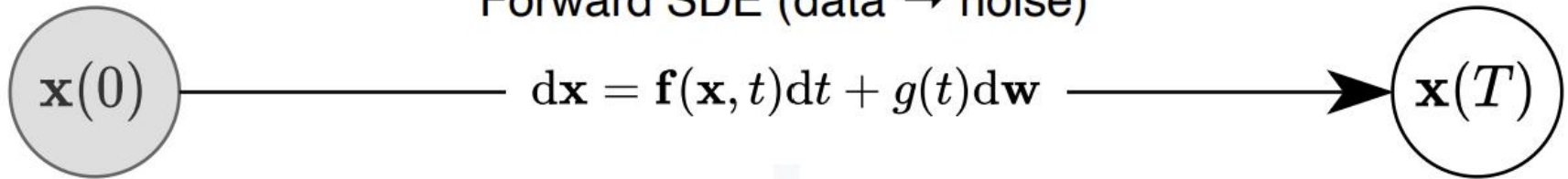
- Score matching

$$\frac{1}{2} E_{p_{data}} \left[\|\nabla_{\mathbf{x}} \log p_{data}(\mathbf{x}) - \nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x})\|_2^2 \right]$$

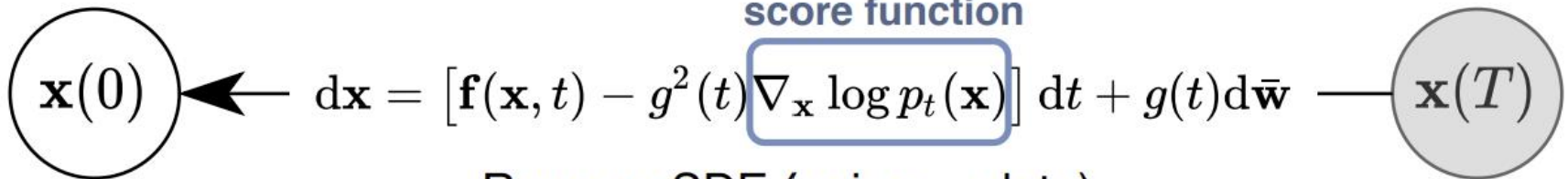
$$\Rightarrow E_{p_{data}} \left[\text{tr}(\nabla_{\mathbf{x}}^2 \log p_{\theta}(\mathbf{x})) + \frac{1}{2} \|\nabla_{\mathbf{x}} \log p_{\theta}(\mathbf{x})\|_2^2 \right] + \text{const}$$

Score-based Generative Model with SDEs

Forward SDE (data $\rightarrow$ noise)



score function



Reverse SDE (noise $\rightarrow$ data)

Diffusion과 Reverse Process를 Stochastic Differential Equation으로 표현

Stochastic Differential Equation (SDE) for Diffusion

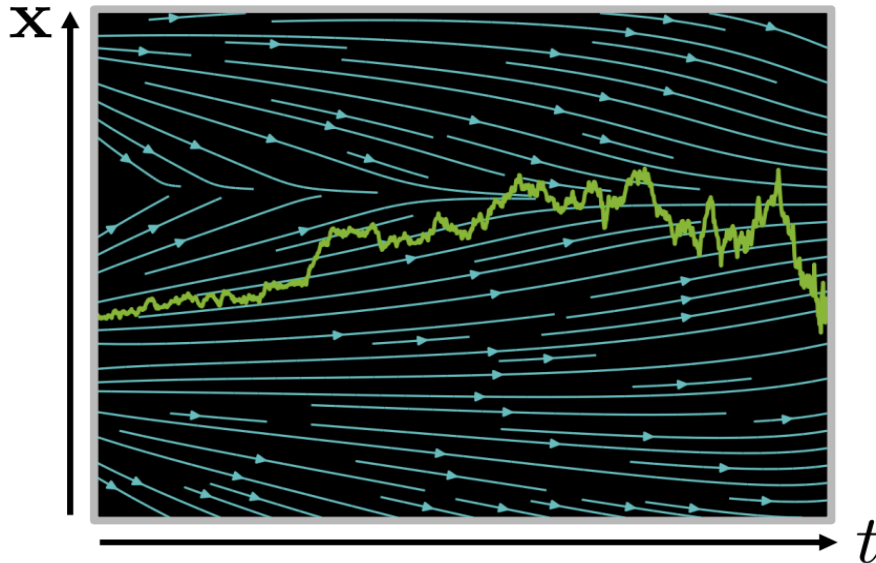
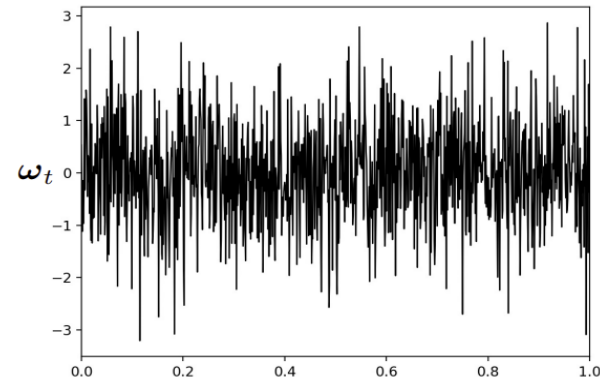
Stochastic Differential Equation (SDE):

$$\frac{d\mathbf{x}}{dt} = \underbrace{\mathbf{f}(\mathbf{x}, t)}_{\text{drift coefficient}} + \underbrace{\sigma(\mathbf{x}, t)\omega_t}_{\text{diffusion coefficient}}$$

$$\left(d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + \sigma(\mathbf{x}, t)d\omega_t \right)$$

(a.k.a Brownian motion)

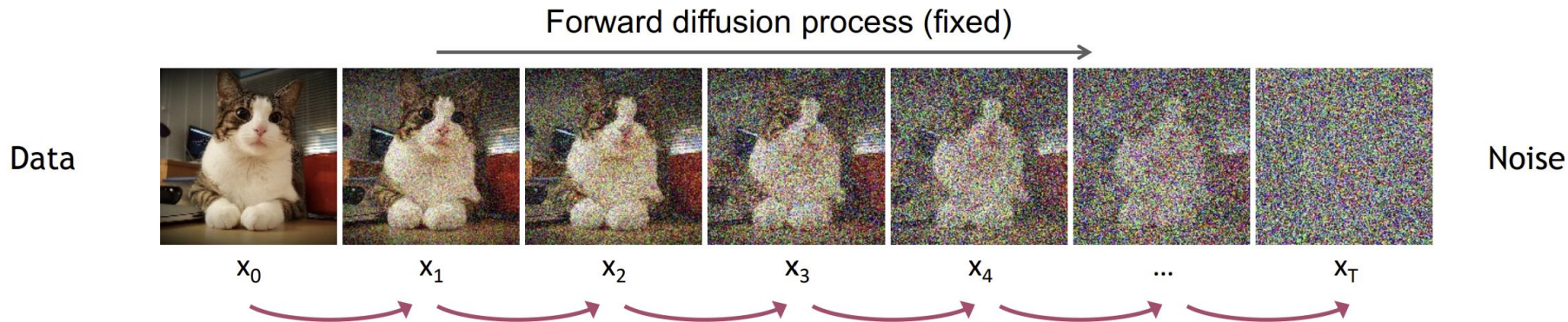
Wiener Process
(Gaussian
White Noise)



$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t + \sigma(\mathbf{x}(t), t)\sqrt{\Delta t}\mathcal{N}(\mathbf{0}, \mathbf{I})$$



Forward Diffusion Process with SDE



$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$

↓ discretization of

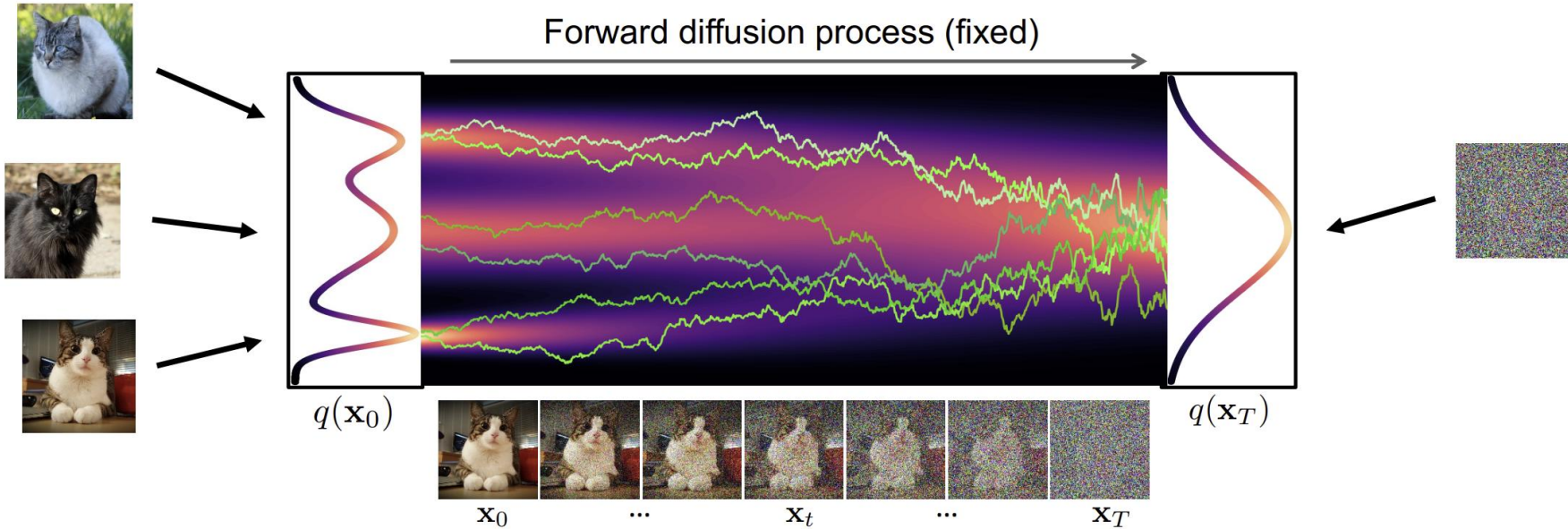
$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)} d\boldsymbol{\omega}_t$$

Stochastic Differential Equation (SDE)

일반적인 Diffusion의 스텝을 무한히 작게 하여 연속적으로 만들면
Diffusion process에 대한 SDE로 표현가능



Forward Diffusion Process with SDE



Forward Diffusion SDE:

$$d\mathbf{x}_t = \underbrace{-\frac{1}{2}\beta(t)\mathbf{x}_t dt}_{\text{drift term (pulls towards mode)}} + \underbrace{\sqrt{\beta(t)} d\omega_t}_{\text{diffusion term (injects noise)}}$$

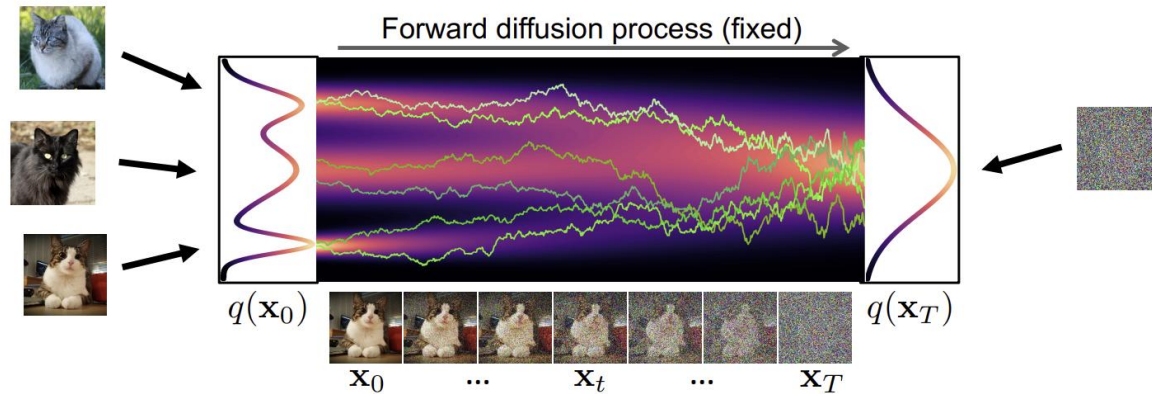
평균

노이즈

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I})$$



Reverse Denoising Process with SDE



Forward Diffusion SDE:

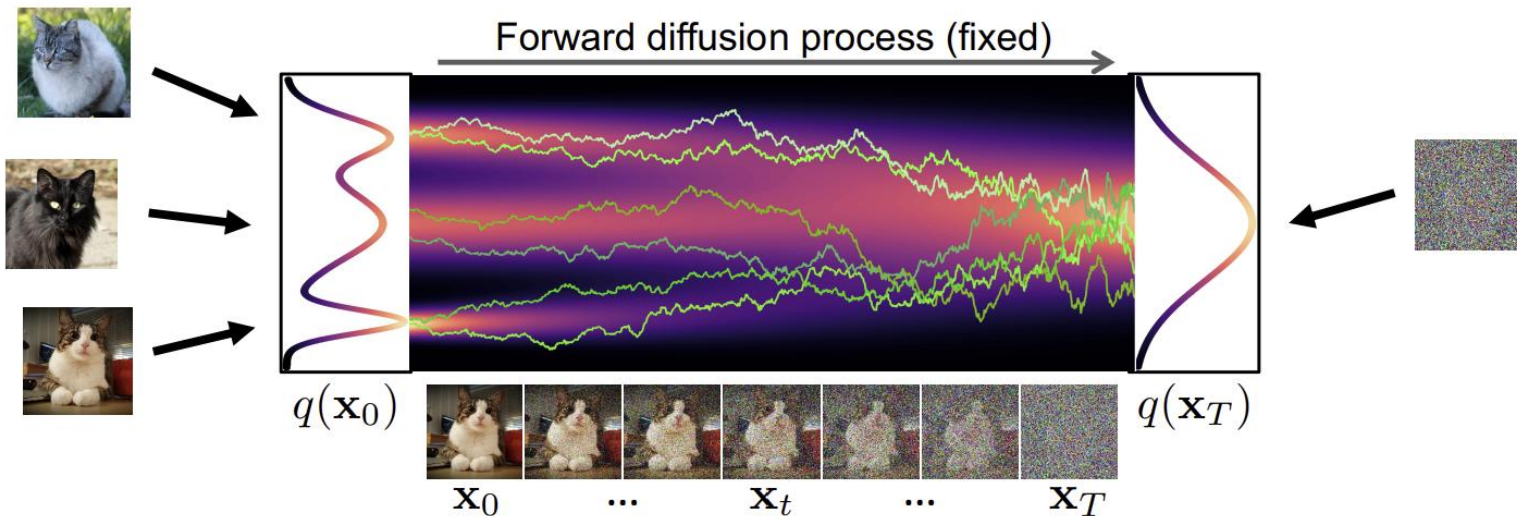
$$d\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t dt + \sqrt{\beta(t)} d\boldsymbol{\omega}_t$$

**Reverse Generative
Diffusion SDE:**

$$d\mathbf{x}_t = \underbrace{\left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t) \underbrace{\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)}_{\text{"Score Function"}} \right]}_{\text{drift term}} dt + \underbrace{\sqrt{\beta(t)} d\bar{\boldsymbol{\omega}}_t}_{\text{diffusion term}}$$



Score Matching



$$\min_{\theta} \underbrace{\mathbb{E}_{t \sim \mathcal{U}(0, T)}}_{\text{diffusion time } t} \underbrace{\mathbb{E}_{\mathbf{x}_t \sim q_t(\mathbf{x}_t)}}_{\text{diffused data } \mathbf{x}_t} \underbrace{\|\mathbf{s}_{\theta}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)\|_2^2}_{\text{score of diffused data (marginal)}}$$

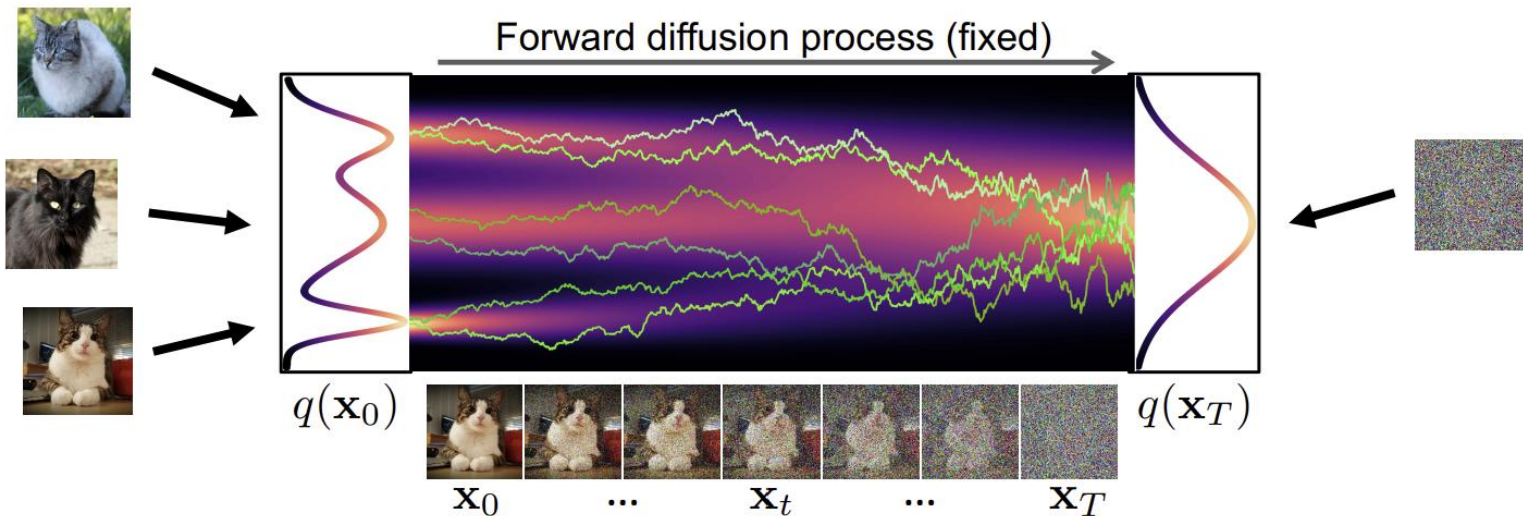
값을 찾기 어려움



Score Matching

$$q_{\sigma}(\tilde{\mathbf{x}} \mid \mathbf{x}) = \mathcal{N}(\tilde{\mathbf{x}} \mid \mathbf{x}, \sigma^2 I)$$

$$\nabla_{\tilde{\mathbf{x}}} \log q_{\sigma}(\tilde{\mathbf{x}} \mid \tilde{\mathbf{x}}) = -(\tilde{\mathbf{x}} - \mathbf{x})/\sigma^2$$



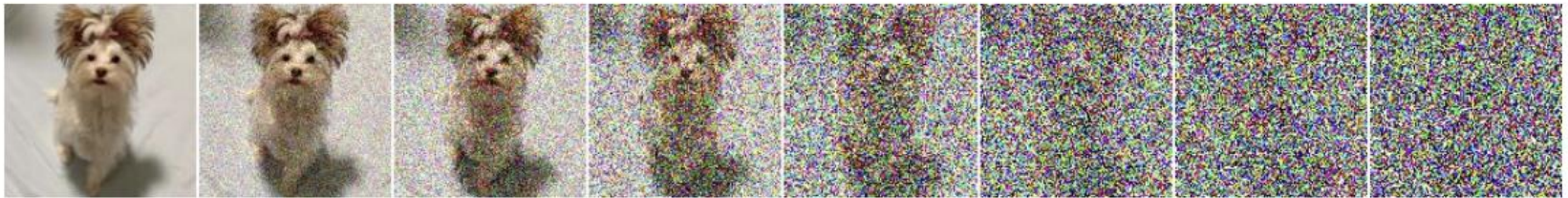
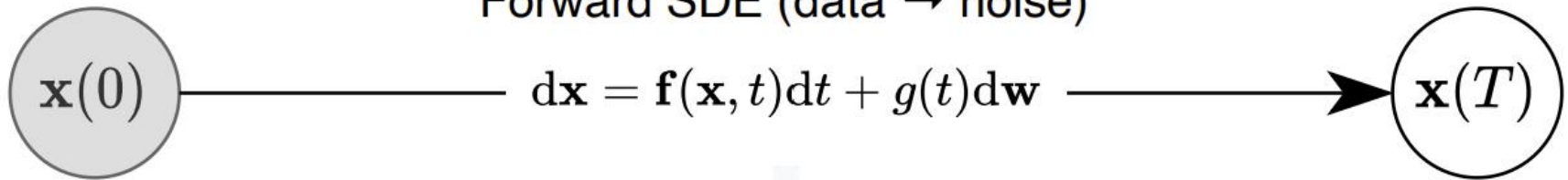
$$\min_{\theta} \underbrace{\mathbb{E}_{t \sim \mathcal{U}(0, T)}}_{\text{diffusion time } t} \underbrace{\mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)}}_{\text{data sample } \mathbf{x}_0} \underbrace{\mathbb{E}_{\mathbf{x}_t \sim q_t(\mathbf{x}_t | \mathbf{x}_0)}}_{\text{diffused data sample } \mathbf{x}_t} \underbrace{\|\mathbf{s}_{\theta}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t | \mathbf{x}_0)\|_2^2}_{\text{neural network score of diffused data sample}}$$

$$\min_{\theta} \mathbb{E}_{t \sim \mathcal{U}(0, T)} \mathbb{E}_{\mathbf{x}_0 \sim q_0(\mathbf{x}_0)} \mathbb{E}_{\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \frac{1}{\sigma_t^2} \|\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)\|_2^2$$

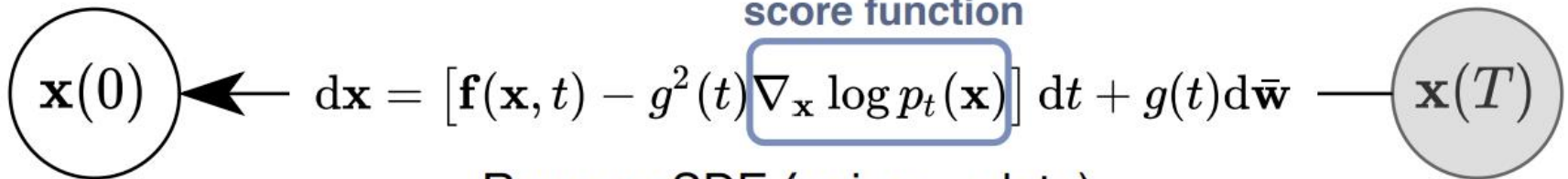
Gaussian 분포 찾기 쉬움

Score-based Generative Model with SDEs

Forward SDE (data $\rightarrow$ noise)



score function



Reverse SDE (noise $\rightarrow$ data)

Diffusion과 Reverse Process를 Stochastic Differential Equation으로 표현

- Diffusion 모델의 일반화
- 다양한 변형과 새로운 손실함수를 제안하는 것이 가능



Variance Exploding vs. Preserving Diffusion

General diffusion SDE

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)dt + g(t)d\mathbf{w}$$

Variance exploding (SMLD)

$$d\mathbf{x} = \sqrt{\frac{d[\sigma^2(t)]}{dt}}d\mathbf{w} \longrightarrow \mathbf{x}_i = \mathbf{x}_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2}\mathbf{z}_{i-1}, \quad i = 1, \dots, N,$$

Variance preserving (DDPM)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)} d\mathbf{w}$$
$$\longrightarrow \mathbf{x}_i = \sqrt{1 - \beta_i}\mathbf{x}_{i-1} + \sqrt{\beta_i}\mathbf{z}_{i-1}, \quad i = 1, \dots, N.$$

Sub-Variance preserving (new!!)

$$d\mathbf{x} = -\frac{1}{2}\beta(t)\mathbf{x} dt + \sqrt{\beta(t)(1 - e^{-2\int_0^t \beta(s)ds})}d\mathbf{w}$$

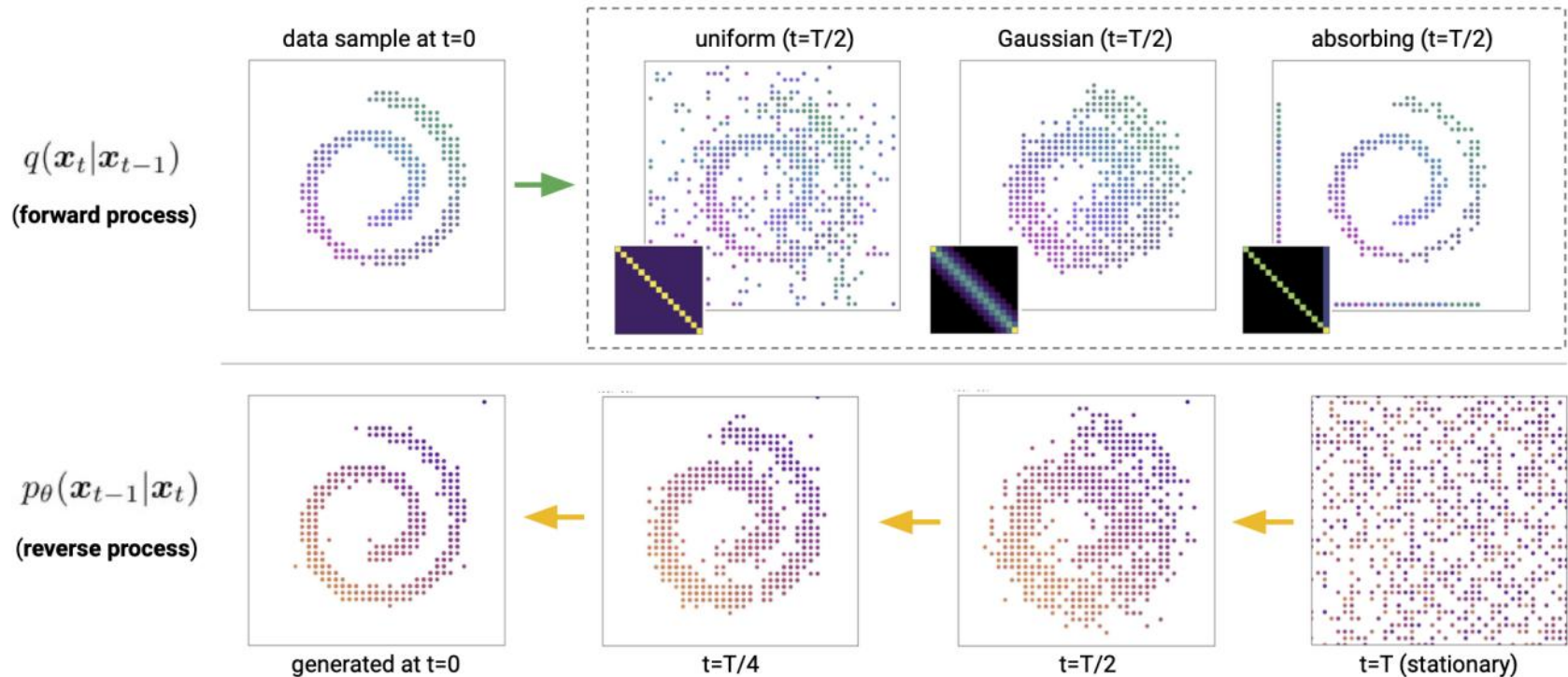
생성모델

D3PM



D3PM: Discrete Denoising Diffusion Probabilistic Model

- DDPM은 continuous 한 확률 분포에 적용
- D3PM: DDPM을 discrete한 확률 분포에 적용
 - 이미지: 각 픽셀의 밝기 단계, 0~254
 - 텍스트: 각 토큰의 선택





Diffusion Process

- 문제 세팅

각 변수 $x_t, x_{t-1} \in 1, \dots, K$

전이확률 $[Q_t]_{ij} = q(x_t = j | x_{t-1} = i)$

확률분포 $q(x_t | x_{t-1}) = \text{Cat}(x_t; p = x_{t-1} Q_t)$

- Forward process

$$q(x_t | x_0) = \text{Cat}(x_t; p = x_0 \bar{Q}_t), \quad \text{with} \quad \bar{Q}_t = Q_1 Q_2 \dots Q_t$$

- Reverse process

실제로 모델을 통해 예측되는 부분

$$p_\theta(x_{t-1} | x_t) \propto \sum_{\tilde{x}_0} q(x_{t-1}, x_t | \tilde{x}_0) \underbrace{\tilde{p}_\theta(\tilde{x}_0 | x_t)}$$

$$q(x_{t-1} | x_t, x_0) = \frac{q(x_t | x_{t-1}, x_0) q(x_{t-1} | x_0)}{q(x_t | x_0)} = \text{Cat} \left(x_{t-1}; p = \frac{x_t Q_t^\top \odot x_0 \bar{Q}_{t-1}}{x_0 \bar{Q}_t x_t^\top} \right)$$



Noise Model

- DDPM에서는 Gaussian 노이즈를 사용하였지만, D3PM에서는 다양한 모델로 Q를 선택하는 것이 가능

- Uniform:
$$[Q_t]_{ij} = \begin{cases} 1 - \frac{K-1}{K}\beta_t & \text{if } i = j \\ \frac{1}{K}\beta_t & \text{if } i \neq j \end{cases}$$

- Absorbing state: 새로운 state인 [MASK]를 가정하고 각 변수를 확률적으로 [MASK]로 만들

$$[Q_t]_{ij} = \begin{cases} 1 & \text{if } i = j = m \\ 1 - \beta_t & \text{if } i = j \neq m \\ \beta_t & \text{if } j = m, i \neq m \end{cases}$$

- Discretized Gaussian

$$[Q_t]_{ij} = \begin{cases} \frac{\exp\left(-\frac{4|i-j|^2}{(K-1)^2\beta_t}\right)}{\sum_{n=-(K-1)}^{K-1} \exp\left(-\frac{4n^2}{(K-1)^2\beta_t}\right)} & \text{if } i \neq j \\ 1 - \sum_{l=0, l \neq i}^{K-1} [Q_t]_{il} & \text{if } i = j \end{cases}$$

- 손실함수

$$\mathbb{E}_{q(\mathbf{x}_0)} [-\log p_\theta(\mathbf{x}_0)] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] =: L$$

- DDPM: Gaussian 노이즈를 더해가는 continuous 한 경우

$$L = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), t \sim \mathcal{U}\{1, T\}, \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[\lambda_t \|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2 \right]$$

- D3PM: 하이브리드 loss를 사용 (L_λ)

$$L_{\text{vb}} = \mathbb{E}_{q(\mathbf{x}_0)} \left[\underbrace{D_{\text{KL}}[q(\mathbf{x}_T|\mathbf{x}_0)||p(\mathbf{x}_T)]}_{L_T} + \sum_{t=2}^T \underbrace{\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [D_{\text{KL}}[q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)||p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)]]}_{L_{t-1}} \right. \\ \left. - \underbrace{\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} [\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]}_{L_0} \right].$$

$$L_\lambda = L_{\text{vb}} + \lambda \mathbb{E}_{q(\mathbf{x}_0)} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} [-\log \tilde{p}_\theta(\mathbf{x}_0|\mathbf{x}_t)].$$

현재 t 에서 원본 데이터를 얼마나 잘 예측할 수 있을지

감사합니다